



Universidad de los Andes

Department of Systems and Computer Engineering

APP REPORT – Part 1, Part 2, Part 3 & Part 4

ISIS 3510 – Mobile App Development

KDE Connect (Android)

Group 32 Sub-Group 1:

Gabriela Carvajal (202111058)

Juan Camilo Nieves (202116708)

Jerónimo A. Pineda Cano (202212778)

2026-1

Repository link:

<https://invent.kde.org/network/kdeconnect-android>

Mirror: <https://github.com/KDE/kdeconnect-android>

Contents

I	Part 1	4
1	Application Description	4
1.1	Identification and General Information	4
1.2	Core functionalities	6
1.3	Architecture and Privacy Philosophy	9
1.4	Relevance and Key Aspects	9
2	Repository Description	9
2.1	Hosting, visibility, and license	9
2.2	Languages and build system	10
2.3	Commits, branches, and tags	11
2.4	Contributors and structure	12
3	Business Questions	12
3.1	Type 2 – Descriptive (data shown in the app)	12
3.2	Type 4 or 5 – Strategic / predictive or prescriptive (revised after feedback)	12
4	Bibliographic reference (Part 1)	13
II	Part 2	14
5	Architecture – Design patterns	14
5.1	Template Method (plugin lifecycle)	14
5.2	Factory (plugin instantiation)	14
5.3	Observer (device and plugin state changes)	15
6	UI/UX – Design analysis	16
6.1	Basic UI/UX components	16
6.2	What we liked	17
6.3	What could be improved	17
6.4	Detailed analysis of three views	17
7	Quality attributes	19
7.1	Security	19
7.2	Maintainability	20
7.3	Testability	20
8	Libraries and dependencies	20
III	Part 3	22
9	Eventual Connectivity (ECn) strategies	22
9.1	G1. Persistent URL outbox in the Share plugin	22

9.2	G2. Aggressive re-discovery on every network event	23
9.3	G3. Rate-limited identity broadcasting and mDNS	23
9.4	G4. Persistent trust via TOFU	23
9.5	A1. File transfer has no resume; partial files are deleted on failure	24
9.6	A2. The outgoing packet queue is lost when the last link disconnects	25
9.7	A3. The outbox pattern is only implemented for URLs, not for text or files	26
9.8	ECn summary	28
10	Caching Strategies	28
10.1	C1. <code>DiskLruCache</code> for album art	29
10.2	C2. In-memory <code>LruCache</code> for already-decoded bitmaps	29
10.3	C3. <code>SharedPreferences</code> as a configuration and trust cache	30
10.4	C4. <code>ConcurrentHashMap</code> -based in-memory caches	30
10.5	Average amount of cache storage	30
11	Memory Management Strategies	31
11.1	M1. <code>SharedPreferences</code> (persistent, private)	31
11.2	M2. SQLite via <code>SQLiteOpenHelper</code> (persistent, private)	31
11.3	M3. File storage (mix of private cache and user-visible public)	32
11.4	M4. In-memory structures with explicit lifecycle (transient, in-process)	33
11.5	Summary of placement decisions	34
12	Threading / Concurrency Strategies	34
12.1	T1. Shared <code>ExecutorService</code> through <code>ThreadHelper</code>	34
12.2	T2. Kotlin coroutines for the per-device send queue and background work	35
12.3	T3. <code>BackgroundJobHandler</code> : a fixed pool with <code>Future</code> management	36
12.4	T4. Dedicated <code>Thread</code> + <code>Looper</code> for the SMS content observer	37
12.5	T5. Lock-free and lock-biased data structures for the UI and worker boundary . .	37
12.6	Main-thread marshalling	38
12.7	Threading summary	38
IV	Part 4	38
13	Methodology and device under test	38
14	Scenario S1 – Cold start and UI baseline	39
14.1	Scenario definition	39
14.2	CPU consumption and threading	40
14.3	Memory management	41
14.4	GPU rendering analysis	41
14.5	Overdraw	42
14.6	Power consumption	43
14.7	Findings	44
15	Scenario S2 – Large file transfer with network failure and leak detection	44
15.1	Scenario definition	44
15.2	CPU consumption and threading	45
15.3	Memory management	45
15.4	GPU rendering analysis	48
15.5	Overdraw	49
15.6	Power consumption	49
15.7	Memory leak in <code>DeviceFragment</code> : detection, root cause and fix	49

15.8 Upstream contribution to KDE Connect	51
15.9 Findings	53
16 Scenario S3 – Screen rotation (configuration change)	54
16.1 Scenario definition	54
16.2 CPU consumption and threading	54
16.3 Memory management	55
16.4 GPU rendering analysis	56
16.5 Overdraw	57
16.6 Power consumption	58
16.7 Findings	58
17 Cross-scenario synthesis: GC, threading and main-thread contention	58
18 Micro-optimisations and detected anti-patterns	59
18.1 MO1. MD5 hashing for cache keys	59
18.2 MO2. Lock-biased containers (<code>CopyOnWriteArrayList</code>) for read-heavy listener lists	60
18.3 MO3. Bounded in-memory cache (<code>LruCache</code>) for decoded bitmaps	60
18.4 MO4. Lazy plugin loading via annotations	61
18.5 MO5. Listener reuse in mDNS discovery	61
18.6 A4. Unbounded cached thread pool (anti-pattern, continues A-numbering from Part 3)	61

Part I

Part 1

1 Application Description

1.1 Identification and General Information

The application selected for this analysis is KDE Connect, an open-source tool developed by the KDE Community. It allows integration and two-way communication between mobile devices and desktop computers through a local network.

The version evaluated in this report is the Android client, identified in the Google Play Store as version 1.35.2, whose latest update was published on February 6, 2026. In addition, the project has native clients for Windows, macOS, Linux (KDE Plasma), and an experimental version for iOS. Regarding its user base, the Google Play Store reports more than 1,000,000 downloads at the time of writing this report.

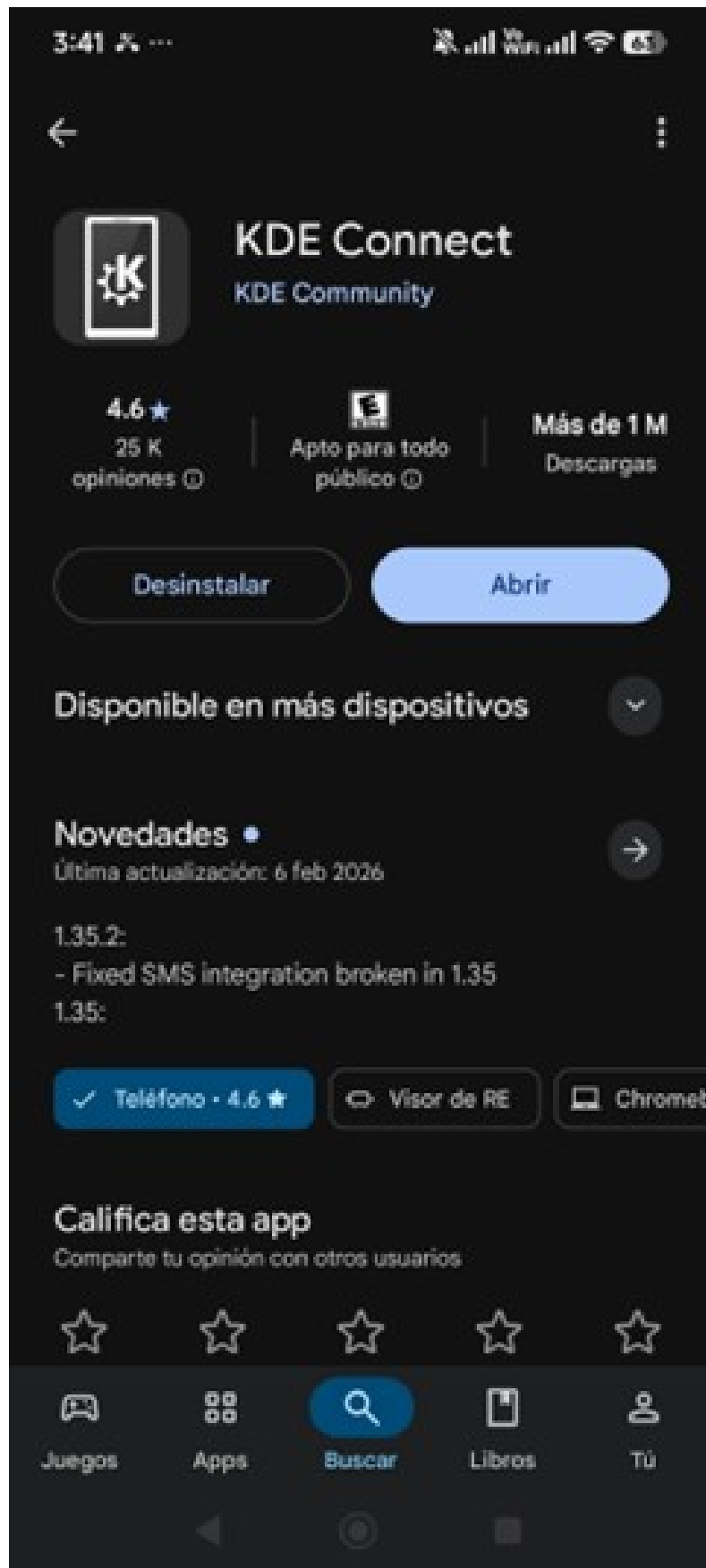


Figure 1: Official KDE Connect page on Google Play (rating 4.6, 25K reviews, 1M+ downloads; version 1.35.2, last update February 6, 2026).

The application is distributed under the GPLv3 license. It is completely free and does not contain ads, in-app purchases, or subscriptions. Its financial support is based on donations to the non-profit organization KDE e.V. and the voluntary work of more than one hundred international contributors. This model, which avoids direct monetization of users, is consistent with the principles of free software and strong privacy supported by the project.

1.2 Core functionalities

KDE Connect offers a modular set of features that are activated through plugins. After installation and testing in a controlled environment (POCO M6 Pro Android phone + Windows 11 computer), the following main capabilities were identified:

- **Automatic and manual pairing:** Devices discover each other on the same WiFi network or through Bluetooth. Manual pairing can also be done using an IP address.
- **Two-way file transfer:** Allows sending and receiving any type of file between the phone and the PC.
- **Shared clipboard:** Text copied on one device is instantly available on the other.
- **Phone notifications:** Phone notifications (calls, SMS, messaging apps) are shown on the desktop.
- **Remote multimedia control:** From the phone, users can play, pause, skip tracks, and adjust the PC's media player volume.
- **Use of the phone as a touchpad:** Especially useful to control the computer when a mouse is not available.
- **Remote commands:** Users can execute predefined actions on the PC (shutdown, restart, lock session) from the phone.
- **Phone location:** Makes the device ring even if it is in silent mode.
- **Plugin system:** Users can enable or disable each feature independently, adapting the app to their needs.

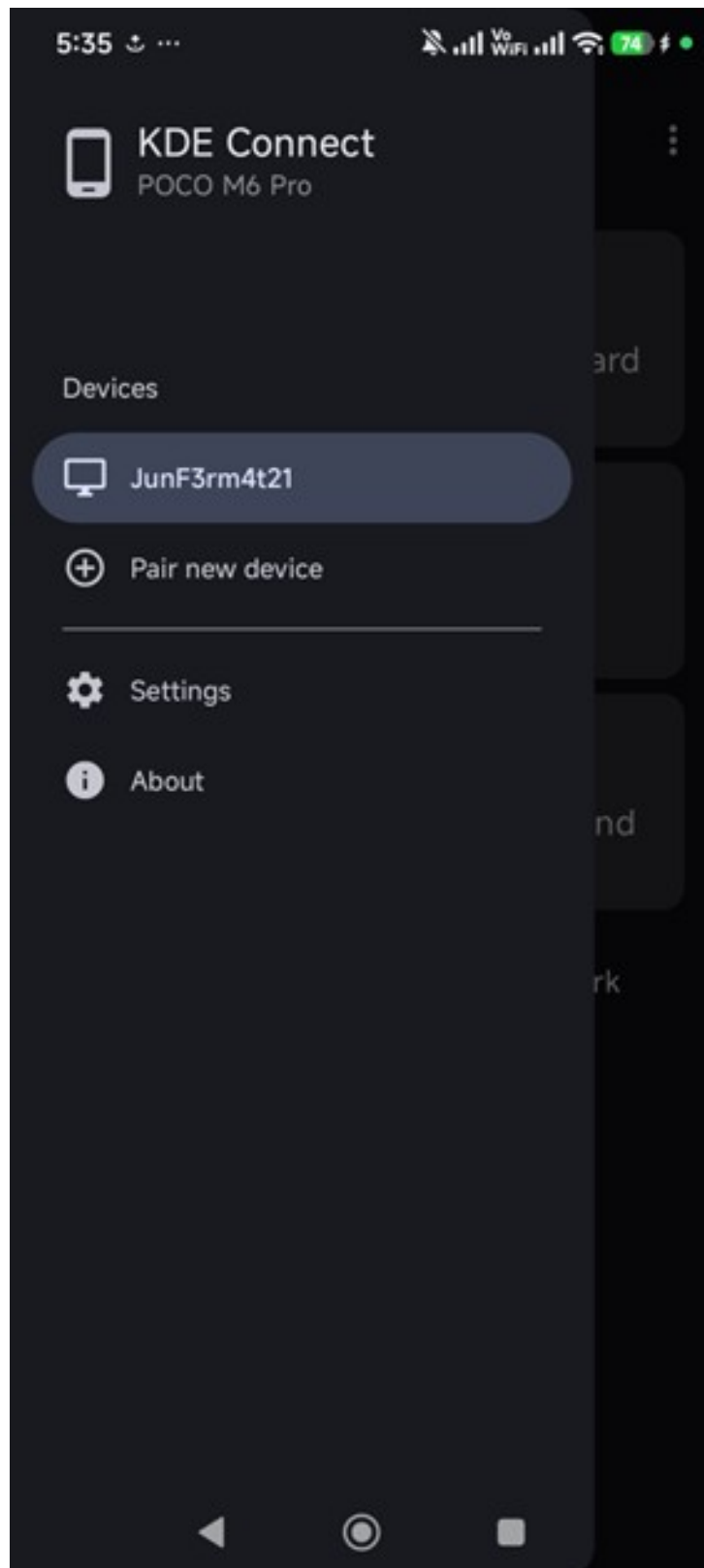


Figure 2: Main interface of KDE Connect on POCO M6 Pro (dark theme). The paired device “JunF3rm4t2t” is shown; menu options include Pair new device, Settings, and About.

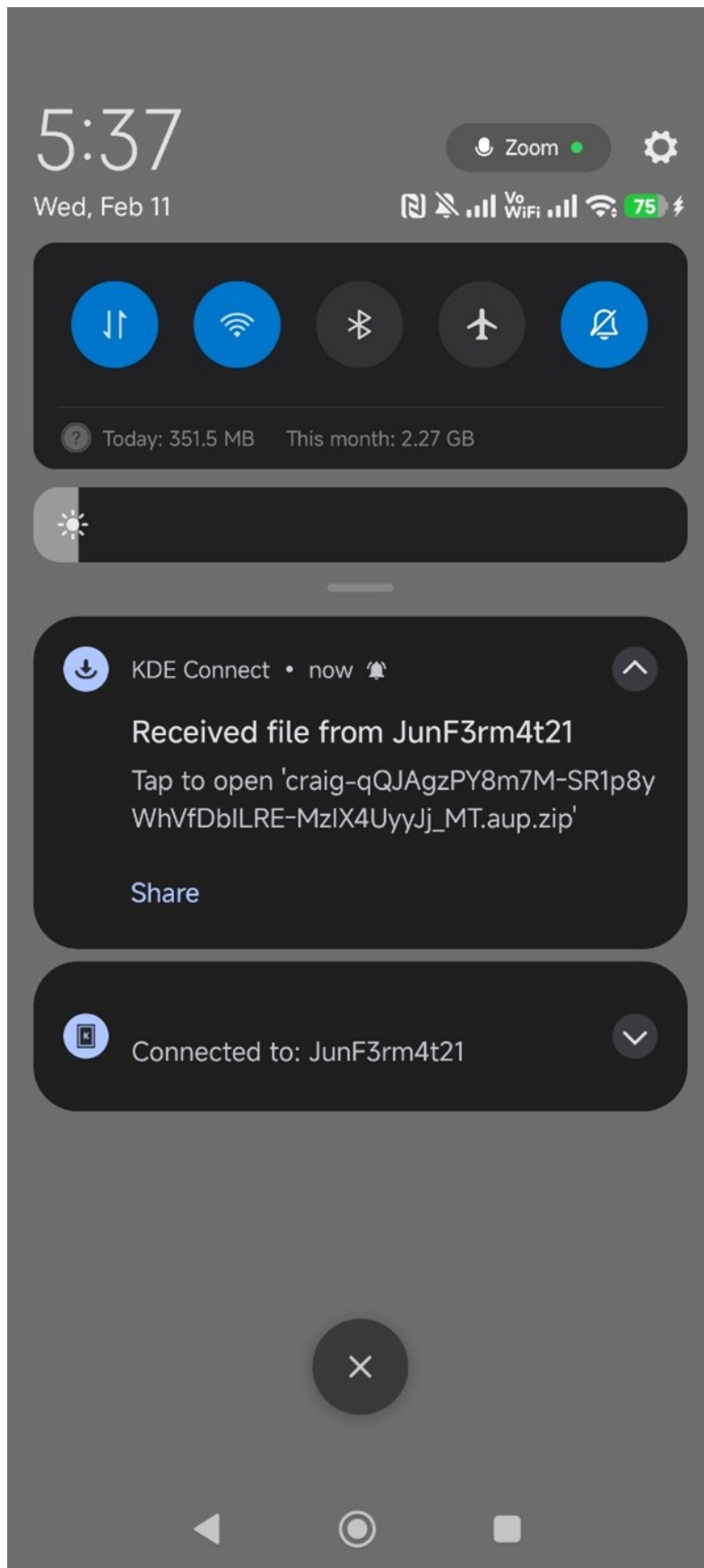


Figure 3: Notification after receiving a file from the paired device (JunF3rm4t21). KDE Connect also shows the persistent “Connected to” status.

1.3 Architecture and Privacy Philosophy

Unlike commercial solutions such as Phone Link (Microsoft) or Intel Unison, which require cloud accounts and send data to external servers, KDE Connect operates only on the local network. Communication between devices is encrypted using TLS, and there is no mandatory telemetry or data collection by KDE. The project’s official website clearly states:

“KDE Connect never sends any information to KDE nor to any third party. KDE Connect sends data from one device to the other directly using the local network, never through the internet, and using end-to-end encryption” [1].

This peer-to-peer architecture not only guarantees user privacy but also allows the application to work without an internet connection, as long as there is a local network (WiFi or Bluetooth).

1.4 Relevance and Key Aspects

KDE Connect is not a new application. Its development began in 2013, and in 2015 it received the Jury’s Award at the Akademy conference. More than a decade later, it still has an active community. The main repository (Android) has more than 4,328 commits and 131 contributors, with almost daily activity.

What makes KDE Connect especially interesting can be summarized in three points:

- **Counter-narrative to data capitalism:** While major manufacturers require users to create accounts and send data to the cloud for device integration, KDE Connect shows that it is possible to have smooth, cross-platform, and ethically responsible integration without giving up privacy.
- **Maturity and constant development:** The recent version 1.35.2 (February 2026) includes improvements in WiFi connection stability and support for Android 15. Its desktop version adds compatibility with native Wayland.
- **Extensible and cross-platform ecosystem:** There are alternative implementations such as GSConnect for the GNOME desktop, a client for SailfishOS, and the experimental iOS version.

2 Repository Description

2.1 Hosting, visibility, and license

The Android app is developed in the repository `kdeconnect-android`. The official project is hosted on KDE’s GitLab at invent.kde.org/network/kdeconnect-android. A mirror is maintained on GitHub at github.com/KDE/kdeconnect-android, which attracts most of the public visibility.

At the time of writing, on GitHub the repository has 1.3k stars and 189 forks. On GitLab (the canonical host), the project shows 48 stars. The license is GPL-2.0 (or later), as stated in the repository; some files use GPL-3.0.

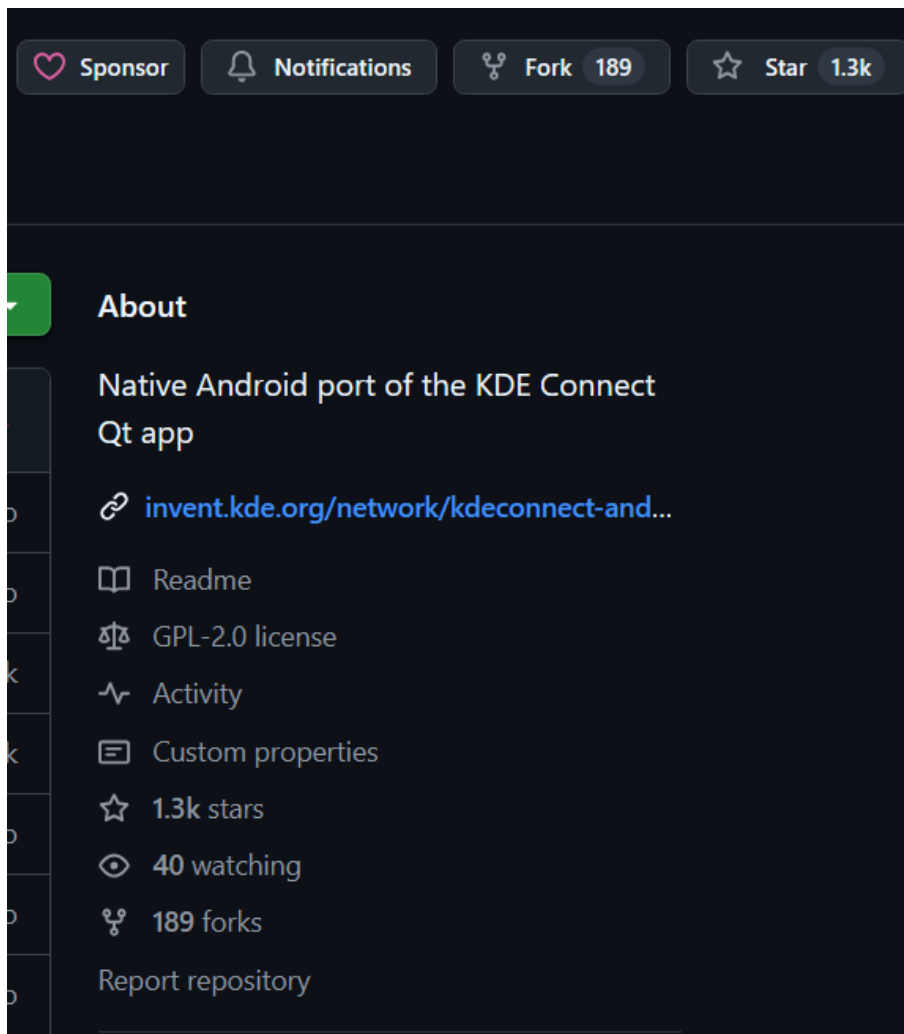


Figure 4: GitHub “About” section: 1.3k stars, 189 forks, GPL-2.0 license, link to [invent.kde.org](https://invent.kde.org/network/kdeconnect-and...).

2.2 Languages and build system

The project uses Kotlin as the main language for new code (UI, Compose, and several plugins) and Java for a significant part of the codebase (plugins, backends, and legacy UI). The build system is Gradle with Kotlin DSL (`build.gradle.kts`). The repository contains roughly 27,500 lines of Kotlin and Java source code under `src/`.

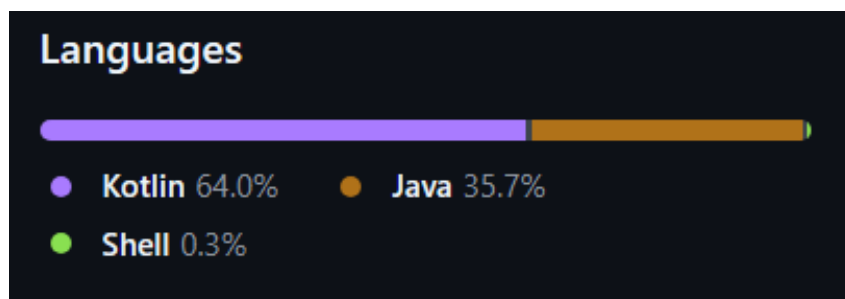


Figure 5: Language distribution: Kotlin 64.0%, Java 35.7%, Shell 0.3%.

2.3 Commits, branches, and tags

The repository has 4,328 commits to date, with frequent activity. There are 23 branches and 128 tags, indicating a long-lived project with structured releases and parallel work (e.g. master, work/* branches).

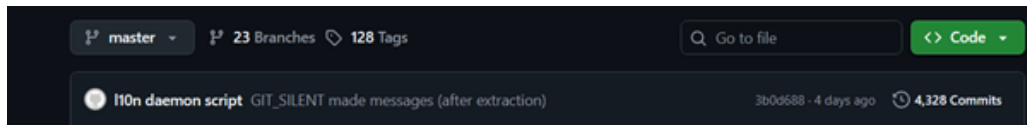


Figure 6: GitHub: master branch, 23 Branches, 128 Tags, 4,328 Commits, latest commit.

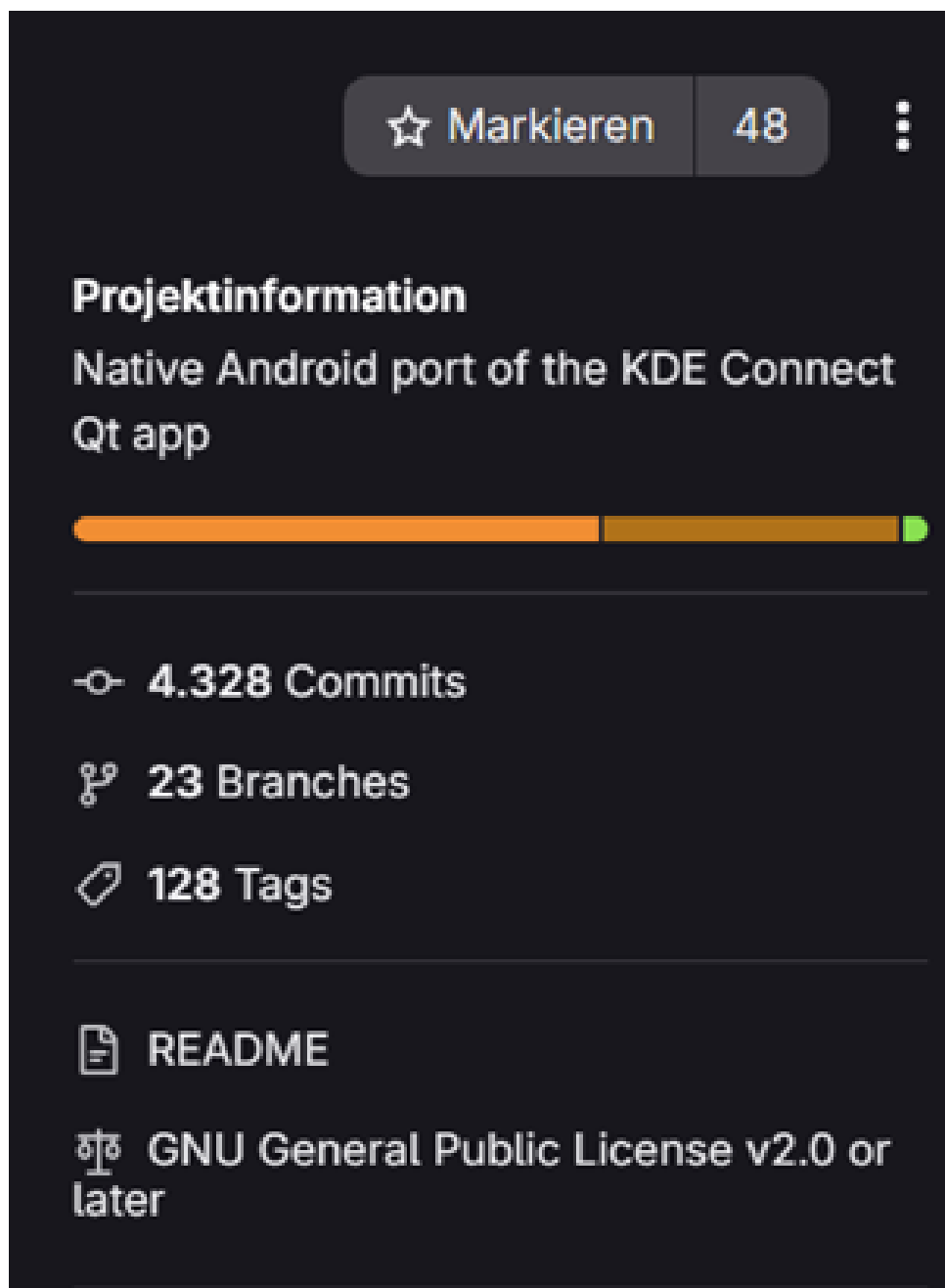


Figure 7: GitLab project information: 48 stars, 4,328 commits, 23 branches, 128 tags, README, GPL v2.0 or later.

2.4 Contributors and structure

The project has over 100 contributors. The main structure under `src/main/java/org/kde/kdeconnect/` includes:

- **ui/** – Activities and fragments (Compose and classic views), including device list, pairing, settings, and plugin UIs.
- **plugins/** – Feature modules (battery, clipboard, share, SFTP, notifications, MPRIS, run command, find my phone, etc.), each implementing the plugin API.
- **backends/** – Connection backends (e.g. LAN discovery and links).

Tests are under `src/test/`. The architecture is plugin-based: each capability is a plugin that can be enabled or disabled by the user.

3 Business Questions

3.1 Type 2 – Descriptive (data shown in the app)

BQ 2.1 – “What is the battery level and charging state of the paired device?”

- **Possible data source:** The BatteryPlugin receives battery data from the remote device via KDE Connect packets (NetworkPacket type battery). On Android, the local device’s battery state is obtained from the system (BatteryManager, ACTION_BATTERY_CHANGED). The plugin holds the latest remote state in DeviceBatteryInfo (level, charging, etc.).
- **How it is displayed in the app:** In the device detail screen (DeviceFragment), when the user selects a paired device, the Battery plugin appears as one of the capabilities. There the user can see the remote device’s battery level and charging status. So the answer to “what is the battery level?” is shown directly in this plugin UI.

BQ 2.2 – “Which devices are currently paired and available?”

- **Possible data source:** The list of paired and visible devices comes from the KdeConnect / BackgroundService state: devices discovered on the LAN (and optionally via Bluetooth) and their pairing status. Each Device object represents one peer (name, id, plugins, connection status).
- **How it is displayed in the app:** On the main screen, the app shows a list of devices (e.g. via DeviceItem and list adapters). Each entry shows the device name and icon; the user can see at a glance which devices are paired and available. If there are none, the app shows the “No devices” message. So the answer to “which devices are paired?” is the very list shown on that screen.

3.2 Type 4 or 5 – Strategic / predictive or prescriptive (revised after feedback)

The type 4/5 question from Part 1 was classified by the grader as type 3 (direct feature improvement). It is replaced below with a clearly strategic/prescriptive question.

BQ 4/5 (revised) – “Should the KDE Connect project prioritize investment in the experimental iOS client versus strengthening the Android/desktop stack, to maximize long-term adoption and sustainability?”

- **Possible data sources:** (1) Download and usage trends on Android (Play Store) and desktop (Flathub, distros). (2) Issue and contribution activity per platform (GitLab/GitHub). (3) Donation or sponsorship data to KDE e.V. (4) Surveys or community feedback on platform demand (forums, Academy).

- **Benefit for the project/app:** This is a resource-allocation and strategy question. Answering it helps decide where to invest limited volunteer/maintainer effort (iOS vs Android/desktop), balancing reach (iOS users) vs ecosystem coherence (Linux/KDE focus) and maintenance cost. It supports sustainability and prioritization at the project level, not only “which feature to build next.”

4 Bibliographic reference (Part 1)

References

- [1] KDE Community, “KDEConnect/PrivacyPolicy,” KDE Community Wiki. [Online]. Available: <https://community.kde.org/KDEConnect/PrivacyPolicy>. [Accessed: 11-Feb-2026].

Part II

Part 2

5 Architecture – Design patterns

The app uses several design and architectural patterns. Below we describe **three** with evidence from the codebase.

5.1 Template Method (plugin lifecycle)

The `Plugin` abstract class defines a common algorithm (load → use → destroy) and leaves specific steps to subclasses. Each plugin overrides hooks such as `onCreate()`, `onDestroy()`, and `onPacketReceived()`.

Evidence – `Plugin.kt` (simplified):

Listing 1: `Plugin.kt` (excerpt)

```
1 abstract class Plugin {
2     open fun onCreate(): Boolean {
3         return true
4     }
5     open fun onDestroy() {}
6     open fun onPacketReceived(np: NetworkPacket): Boolean {
7         return false
8     }
9     abstract val supportedPacketTypes: Array<String>
10    abstract val outgoingPacketTypes: Array<String>
11 }
```

Concrete plugins (e.g. `BatteryPlugin`) override these methods. Example from `BatteryPlugin.kt`:

Listing 2: `BatteryPlugin.kt` (excerpt)

```
1 @LoadablePlugin
2 class BatteryPlugin : Plugin() {
3     override val displayName: String
4         get() = context.resources.getString(R.string.pref_plugin_battery)
5     override val description: String
6         get() = context.resources.getString(R.string.
7             pref_plugin_battery_desc)
8     override val supportedPacketTypes = arrayOf(PACKET_TYPE_BATTERY)
9     override val outgoingPacketTypes = arrayOf(PACKET_TYPE_BATTERY)
10 }
```

So the “template” is the `Plugin` lifecycle; each plugin fills in the steps (supported types, UI, packet handling).

5.2 Factory (plugin instantiation)

`PluginFactory` is responsible for creating plugin instances by key and for a given `Device`. It centralizes the mapping from capability/plugin key to concrete `Plugin` class.

Evidence – `PluginFactory.kt`:

Listing 3: PluginFactory.kt (excerpt)

```

1 object PluginFactory {
2     annotation class LoadablePlugin
3
4     private var pluginInfo: Map<String, PluginInfo> = mapOf()
5
6     fun initPluginInfo(context: Context) {
7         pluginInfo = com.albertvaka.classindexksp.LoadablePlugin
8             .asSequence()
9             .map { it.java.getDeclaredConstructor().newInstance() as
10                 Plugin }
11             .onEach { it.setContext(context, null) }
12             .associate { Pair(it.pluginKey, PluginInfo(it)) }
13     }
14
15     fun instantiatePluginForDevice(context: Context,
16         pluginKey: String, device: Device): Plugin? {
17         try {
18             val plugin = pluginInfo[pluginKey]?.instantiableClass
19                 ?.getDeclaredConstructor()?.newInstance()
20                 ?.apply { setContext(context, device) }
21             return plugin
22         } catch (e: Exception) {
23             Log.e("PluginFactory", "Could not instantiate: $pluginKey",
24                 e)
25             return null
26         }
27     }
28 }

```

Plugin creation is centralized in the Factory; callers use `instantiatePluginForDevice(context, pluginKey, device)` instead of instantiating plugin classes directly.

5.3 Observer (device and plugin state changes)

The app uses observer-style callbacks so the UI and other components react to device list and plugin changes without the core logic depending on the UI.

Evidence – Device.kt:

Listing 4: Device.kt (listeners)

```

1 fun interface PluginsChangedListener {
2     fun onPluginsChanged(device: Device)
3 }
4
5 private val pluginsChangedListeners =
6     CopyOnWriteArrayList<PluginsChangedListener>()
7
8 fun addPluginsChangedListener(listener: PluginsChangedListener) =
9     pluginsChangedListeners.add(listener)
10 fun removePluginsChangedListener(listener: PluginsChangedListener) =
11     pluginsChangedListeners.remove(listener)

```

Device notifies these listeners when plugins change. The UI subscribes and refreshes. Example in `DeviceFragment.kt`:

Listing 5: DeviceFragment.kt (usage)


```

1 private val pluginsChangedListener = PluginsChangedListener {
2     mActivity?.runOnUiThread { refreshUI() }
3 }
4 device?.addPluginsChangedListener(pluginsChangedListener)
5 // ... in onDestroyView:
6 device?.removePluginsChangedListener(pluginsChangedListener)

```

Similarly, `KdeConnect` uses `DeviceListChangedCallback` for device list updates, and `PairingCallback` for pairing events. Together, these form an Observer-style design.

6 UI/UX – Design analysis

6.1 Basic UI/UX components

This section was expanded after Part 2 feedback to be specific about the exact colors and typography sources.

Color. The app relies on Material 3 and, crucially, on **dynamic colors** on Android 12+. There is *one* hard-coded color in the whole resource tree, `res/values/colors.xml`: the “KDE blue” `#3daee9` used only as `launcher_background` for the adaptive launcher icon. Everything else is derived at runtime from the system color scheme. The comment in `colors.xml` is explicit:

Listing 6: `res/values/colors.xml` (full file)

```

1 <resources>
2     <color name="launcher_background">#3daee9</color>
3     <!-- Whenever possible, don't hardcode colors here and rely
4         on the default ones (eg: from the system or user theme) -->
5 </resources>

```

The base theme is declared in `res/values/styles.xml` with parent `Theme.Material3.DynamicColors.DayNight.NoActionBar`, which on Android 12+ resolves primary/secondary/tertiary/surface tokens from the user’s wallpaper-derived palette, and on older releases falls back to Material 3’s default dark/light schemes. The Compose surface (`ui/compose/KdeTheme.kt`) does the same explicitly:

Listing 7: `KdeTheme.kt`: dynamic color scheme selection

```

1 val colorScheme = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
2     when (isSystemInDarkTheme()) {
3         true -> dynamicDarkColorScheme(context)
4         false -> dynamicLightColorScheme(context)
5     }
6 } else {
7     when (isSystemInDarkTheme()) {
8         true -> darkColorScheme()
9         false -> lightColorScheme()
10    }
11 }
12 MaterialTheme(colorScheme = colorScheme, content = content)

```

Other custom styles defined in `styles.xml` are all Material 3 derivatives: `KdeConnectCardStyle` extends `Widget.Material3.CardView.Elevated`, `KdeConnectDialogStyle` extends `ThemeOverlay.Material3.MaterialAlertDialog`, and `KdeConnectButton.IconButton` extends `Widget.Material3.Button.IconButton.Filled`. So on the POCO M6 Pro used for testing

(MIUI / HyperOS over Android 14) the primary/surface colors visible in Figures 2 and 3 are the device’s dynamic accent, not any color hard-coded by KDE Connect.

Typography. The project has **no** `res/font/` directory and declares no `android:fontFamily` in the base theme, so every text style resolves to the platform default: **Roboto** on Android ≤ 11 and the system default typeface on Android 12+ (Roboto on stock AOSP/Pixel, Google Sans Text on some OEM builds). Text hierarchy comes from two places:

- **Classic Views (XML layouts):** the app references `system/AppCompat` `textAppearance` attributes rather than raw sizes. Examples from the layout tree: `?android:attr/textAppearanceMedium` in `list_item_device_entry.xml`, `mpris_control.xml`, `view_pair_request.xml`; `?android:attr/textAppearanceSmall` in `list_item_entry.xml`; `?attr/textAppearanceHeadline6` in `activity_about_kde.xml` and `nav_header.xml`; `?attr/textAppearanceBody2` in `nav_header.xml`. Only a handful of places hard-code a size, such as `AboutButton.TextView` (`android:textSize="12sp"`).
- **Compose surfaces:** text uses the Material 3 type system. For instance, `plugins/presenter/PresenterActivity.kt` writes `style = MaterialTheme.typography.bodyLarge`, which resolves to Material 3’s default Body/Large scale (16sp, regular weight) on top of whichever typeface the device provides.

Spacing and hierarchy (not typography per se) come from `res/values/dimens.xml`:
`activity_horizontal_margin=16dp`, `activity_vertical_margin=16dp`,
`view_default_padding=16dp`, `fab_margin=16dp`, `nav_header_height=112dp`,
`key_height=48dp`.

Design metaphor and components. The main metaphor is a **list of devices** (paired or discoverable); selecting one opens a **detail screen** where **plugin cards** represent capabilities. Navigation is handled by a **drawer** hosted by `MainActivity` for switching devices and reaching Settings/About. Building blocks: Material 3 cards (`CardView`), `IconButton`, `MaterialAlertDialog`, AndroidX Preference screens, and Compose for newer views (`ui/compose/KdeTheme.kt`, `KdeTopAppBar.kt`, `Buttons.kt`, `TextFields.kt`).

6.2 What we liked

- Clear separation between “list of devices” and “current device + plugins.”
- Day/night and dynamic color support improve consistency with the system.
- Plugin-based UI (one card per capability) makes features discoverable and consistent.
- Use of Compose in parts of the app without breaking the existing View-based flows.

6.3 What could be improved

- Onboarding for first-time users (what is pairing, why trust a network) could be more guided.
- Some plugin UIs are still in legacy Views; gradual migration to Compose could unify look and maintainability.
- Empty states (e.g. no devices, no plugins enabled) could be more informative (short explanation + primary action).

6.4 Detailed analysis of three views

As requested in the Part 2 feedback, each view is now accompanied by a screenshot captured on the POCO M6 Pro.

View 1 – Main activity (device list / drawer). `MainActivity` hosts the navigation drawer and the main content (list of devices or “add device”). The drawer lists the current device name, “Add device,” and paired devices; bottom entries are Settings and About. Content area shows either `DeviceFragment` (for a selected device) or `PairingFragment` (add device). This view is the hub for switching context and accessing global settings. *Improvement:* the drawer could show connection status (e.g. “Connected” / “Unreachable”) per device. The screenshot in Figure 2 (reused from Section 1.2) shows this view with the drawer open.

View 2 – Device detail (`DeviceFragment`). `DeviceFragment` shows the selected device and its plugins. It uses a mix of Compose (e.g. `Card`, `MaterialTheme`) and `ViewBinding`. For each loaded plugin it shows a card with icon, name, and actions. Pairing state (pending, paired, unreachable) is also shown. *Improvement:* grouping plugins (e.g. “Media,” “Files,” “System”) could reduce visual load on devices with many plugins.

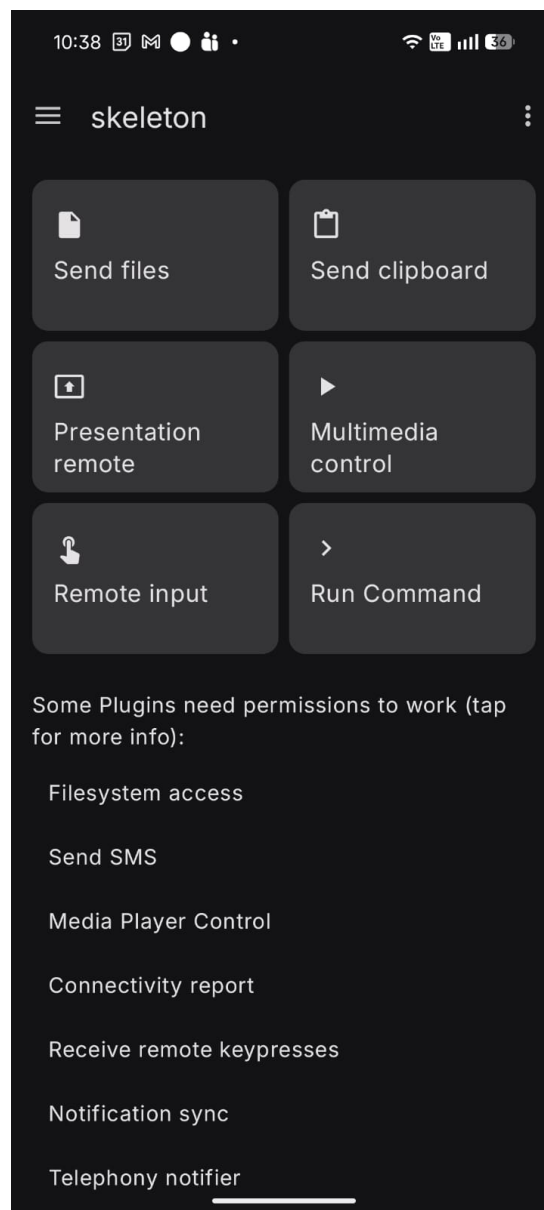


Figure 8: View 2 – `DeviceFragment`: plugin cards for the paired desktop (send files, clipboard, run command, etc.).

View 3 – Pairing (PairingFragment). `PairingFragment` is shown when there are no paired devices or when the user chooses “Add device.” It lists discoverable devices and explains pairing (including messages for no WiFi, duplicate names, or untrusted networks). *Improvement:* a short first-run wizard (one-time explanation of pairing and security) could reduce confusion.

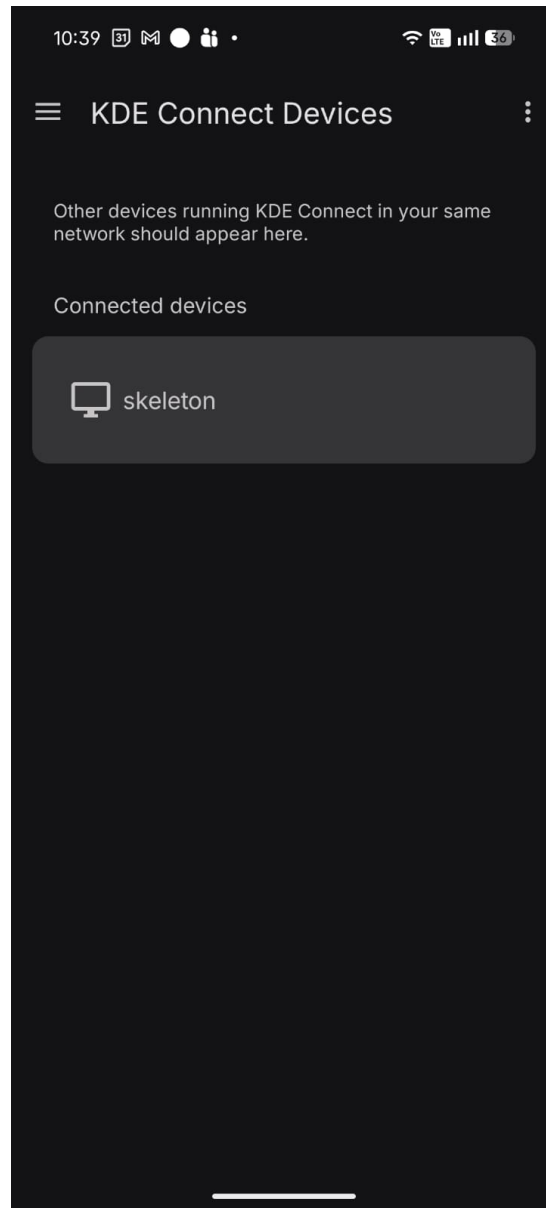


Figure 9: View 3 – `PairingFragment`: list of discoverable devices with pairing guidance.

7 Quality attributes

7.1 Security

The app is designed so that data stays between the two devices and is not sent to the cloud. Communication uses TLS (`SslHelper`). Trust is established by pairing and storing the peer’s certificate (TOFU); `TrustedDevices` and device storage in settings support this. Permissions are requested per plugin (`Plugin.checkRequiredPermissions()`, `PermissionsAlertDialogFragment`). So security is addressed through **encryption in transit, device identity and trust, and least-privilege permissions** per feature.

7.2 Maintainability

The **plugin architecture** (Template Method + Factory) keeps each capability in a single plugin. Adding or changing a feature usually touches one plugin and the shared `Plugin/PluginFactory` contract. The codebase is split into `ui/`, `plugins/`, `backends/`, and `helpers/`, which supports separation of concerns. Kotlin and Compose are used in new code while legacy Java/Views remain, allowing incremental evolution.

7.3 Testability

The project includes unit tests that target core logic and plugins: `DeviceTest`, `DeviceStatsTest`, `NetworkPacketTest`, and plugin tests (e.g. `BatteryPluginTest`, `ClipboardPluginTest`, `PingPluginTest`) under `src/test/`. Dependencies like Mockk and Robolectric allow mocking and Android-independent tests. `@VisibleForTesting` and test-friendly constructors are used where needed. So **testability** is addressed through focused unit tests, mocking, and test-oriented APIs.

8 Libraries and dependencies

The main libraries and their roles (from `build.gradle.kts` and `gradle/libs.versions.toml`):

Library / dependency	Purpose
AndroidX Compose (Material3, Icons, Activity, ConstraintLayout)	Modern UI: Compose screens, Material 3, icons, layout.
AndroidX Core, AppCompat, Preference, RecyclerView, DocumentFile	Core utilities, preferences, lists, file access.
AndroidX Lifecycle (ViewModel, Runtime, Extensions)	Lifecycle-aware components and ViewModels.
Google Android Material	Material components for legacy View-based UI.
Kotlin stdlib, Coroutines (core + android)	Kotlin runtime and async work (coroutines).
Apache SSHD (core, sftp, scp, mina) + Apache Mina	SFTP/SCP server for “filesystem access” plugin.
Bouncy Castle (bcpkix-jdk15on)	Crypto for SSL/TLS certificate generation (pairing).
android-smsmms (org.kde.invent.sredman)	Handling of SMS/MMS for the telephony/SMS plugin.
DiskLruCache (Jake Wharton)	Caching (e.g. album art in MPRIS plugin).
SLF4J (api, handroid)	Logging facade; handroid for Android-backed logging.
RxJava 2 + Reactive Streams	Reactive streams (e.g. device controls / media).
Commons IO, Collections4, Lang3	File/collection utilities and helpers.
Univocity parsers	CSV/parsing.
ClassIndex KSP (com.github.albertvaka)	Discover <code>@LoadablePlugin</code> classes at compile time.
coreLibraryDesugaring (desugar_jdk_libs_nio)	Backport of NIO APIs for minSdk < 26.
JUnit, Mockk, Robolectric, jsonassert	Unit and Android tests, mocking, JSON assertions.

Table 1: Main libraries and their purposes.

The app combines Android/Compose for UI, Kotlin/coroutines for concurrency, SSHD for file access, Bouncy Castle for security, plugin discovery (KSP) for extensibility, and testing libraries for quality.

Part III

Part 3

9 Eventual Connectivity (ECn) strategies

KDE Connect is, by design, an eventually-connected app: the paired desktop may be unreachable for seconds, minutes or hours while the phone remains usable. We read every packet-producing use case in the repository and classified what we found into two groups: good strategies that make the app behave well across disconnects, and antipatterns that cause silent data loss.

9.1 G1. Persistent URL outbox in the Share plugin

Going through `plugins/share/`, we noticed that when the user shares an `http/https` URL from Android while the target desktop is offline, KDE Connect does *not* drop the URL: it is stored in the application `SharedPreferences` under a per-device key and is replayed on the next `onCreate()` of the plugin (i.e. when the peer becomes reachable again and the plugin is instantiated). This is a textbook outbox pattern, and the only solid ECn primitive we could find in the codebase.

Evidence – `ShareActivity.java` and `SharePlugin.java`:

Listing 8: `ShareActivity.java`: queue URL when device is not reachable

```
1 private Unit deviceClicked(Device device, boolean intentHasUrl, Intent
  intent) {
2     SharePlugin plugin = KdeConnect.getInstance()
3         .getDevicePlugin(device.getDeviceId(), SharePlugin.class);
4     if (intentHasUrl && !device.isReachable()) {
5         // Store the URL to be delivered once device becomes online
6         storeUrlForFutureDelivery(device,
7             intent.getStringExtra(Intent.EXTRA_TEXT));
8     } else if (plugin != null) {
9         plugin.share(intent);
10    }
11    finish();
12    return Unit.INSTANCE;
13 }
14
15 private void storeUrlForFutureDelivery(Device device, String url) {
16     Set<String> oldUrlSet = mSharedPrefs.getStringSet(
17         KEY_UNREACHABLE_URL_LIST + device.getDeviceId(), null);
18     Set<String> newUrlSet = new HashSet<>();
19     newUrlSet.add(url);
20     if (oldUrlSet != null) newUrlSet.addAll(oldUrlSet);
21     mSharedPrefs.edit().putStringSet(
22         KEY_UNREACHABLE_URL_LIST + device.getDeviceId(), newUrlSet).
23         apply();
24 }
```

Listing 9: `SharePlugin.java`: drain the outbox on reconnect

```
1 private void deliverPreviouslySentIntents() {
2     Set<String> currentUrlSet = mSharedPrefs.getStringSet(
3         KEY_UNREACHABLE_URL_LIST + device.getDeviceId(), null);
4     if (currentUrlSet != null) {
5         for (String url : currentUrlSet) {
6             Intent intent = Intent.parseUri(url, 0);
```

```

7         intent.putExtra(Intent.EXTRA_TEXT, url);
8         share(intent);
9     }
10    mSharedPreferences.edit().putStringSet(
11        KEY_UNREACHABLE_URL_LIST + device.getId(), null).apply()
12    }
13 }

```

We consider this strategy good because the state is persistent (it survives app kill), it is per-device (multiple peers do not conflict since the key is `KEY_UNREACHABLE_URL_LIST + deviceId`), and it is drained on reconnect automatically (`onCreate` of the plugin runs when `Device.reloadPluginsFromSettings` re-creates it after the link is re-established).

9.2 G2. Aggressive re-discovery on every network event

In `BackgroundService.kt` we found that the service registers a `NetworkCallback` for every non-cellular transport (Wi-Fi, Ethernet, VPN, Bluetooth, USB). On the `onAvailable` callback it triggers `onNetworkChange(network)`, which in turn asks every `BaseLinkProvider` to re-broadcast identity packets. This is what made re-pairing automatic in our tests after we toggled Wi-Fi on the phone or moved between networks.

Evidence – `BackgroundService.kt`:

Listing 10: `BackgroundService.kt`: react to connectivity events

```

1 connectivityManager?.registerNetworkCallback(
2     networkRequestBuilder.build(),
3     object : NetworkCallback() {
4         override fun onAvailable(network: Network) {
5             connectedToNonCellularNetwork.postValue(true)
6             onNetworkChange(network) // -> LanLinkProvider.
              broadcastUdpIdentityPacket
7         }
8         override fun onLost(network: Network) {
9             connectedToNonCellularNetwork.postValue(false)
10        }
11    })

```

9.3 G3. Rate-limited identity broadcasting and mDNS

In `LanLinkProvider.java` we found that UDP identity broadcasts are debounced (`delayBetweenBroadcasts = 200ms`) and incoming handshakes are rate-limited (`MILLIS_DELAY_BETWEEN_CONNECTIONS_TO_SAME_DEVICE = 1s`, up to `MAX_RATE_LIMIT_ENTRIES = 255` recent peers tracked in two `ConcurrentHashMaps`). Together with `MdnsDiscovery`, this means re-connection after a transient drop is near-instant and does not flood the network. We confirmed this observationally: after toggling Wi-Fi off and on, the persistent notification went back to “Connected to: *JunF3rm4t21*” within one or two seconds.

9.4 G4. Persistent trust via TOFU

Pair state and the peer’s X.509 certificate are stored in `SharedPreferences` by `TrustedDevices` and `DeviceInfo.saveInSettings()`. After any disconnect or app kill, `KdeConnect.loadRememberedDevicesFromSettings()` rebuilds the `Device` object without asking the user to re-pair. So ECn recovery does not cost UX friction, which we verified by

force-stopping the app from Android settings and relaunching it: the paired desktop re-appeared in the drawer automatically.

9.5 A1. File transfer has no resume; partial files are deleted on failure

The code that receives files, `CompositeReceiveFileJob.java`, streams each payload from the peer’s TCP socket into a freshly created `DocumentFile`. If the socket dies mid-transfer (peer goes offline, Wi-Fi drops, device locks), the exception path *deletes the partial file* and surfaces a “Failed to receive X files” notification. We consider this an antipattern because on unstable Wi-Fi (common on mobile) a 300 MB video transferred 95% will simply evaporate, forcing the user to restart from byte 0.

Evidence – `CompositeReceiveFileJob.java`:

Listing 11: `CompositeReceiveFileJob.java`: partial file is discarded

```
1 long received = receiveFile(inputStream, outputStream);
2 currentNetworkPacket.getPayload().close();
3
4 if (received != currentNetworkPacket.getPayloadSize()) {
5     fileDocument.delete();                                // <-- lost
6     bytes
7     if (!isCancelled()) {
8         throw new RuntimeException("Failed to receive: " +
9             currentFileName
10             + " received:" + received + " bytes, expected: "
11             + currentNetworkPacket.getPayloadSize() + " bytes");
12     }
13 }
```

The KDE Connect protocol already carries `numberOfFiles` and `totalPayloadSize`, so resume would be implementable by keeping the partial blob, recording the byte offset, and asking the peer to reopen a payload from that offset. Google’s Nearby Share and `rsync` both show that this UX is worth the effort.

Reproduction. To reproduce this antipattern in our test environment (see Figure 10) we followed these steps:

1. We paired the phone with the desktop as in Part 1.
2. From the desktop, we sent a large file (≥ 200 MB, a video) to the phone.
3. When the phone’s progress notification passed around 50%, we enabled Airplane mode on the phone for 2–3 seconds and then disabled it.
4. We observed the KDE Connect notification flip to “Failed to receive 1 of 1 files from *JunF3rm4t21*”; we opened Files / Downloads and confirmed that the partial file was not there.

Proposed fix. Keep partial writes and record (`peerId`, `fileKey`, `bytesReceived`) in `SharedPreferences`; on reconnect, send an `open` or `resume` request with the byte offset before the peer re-queues the file.

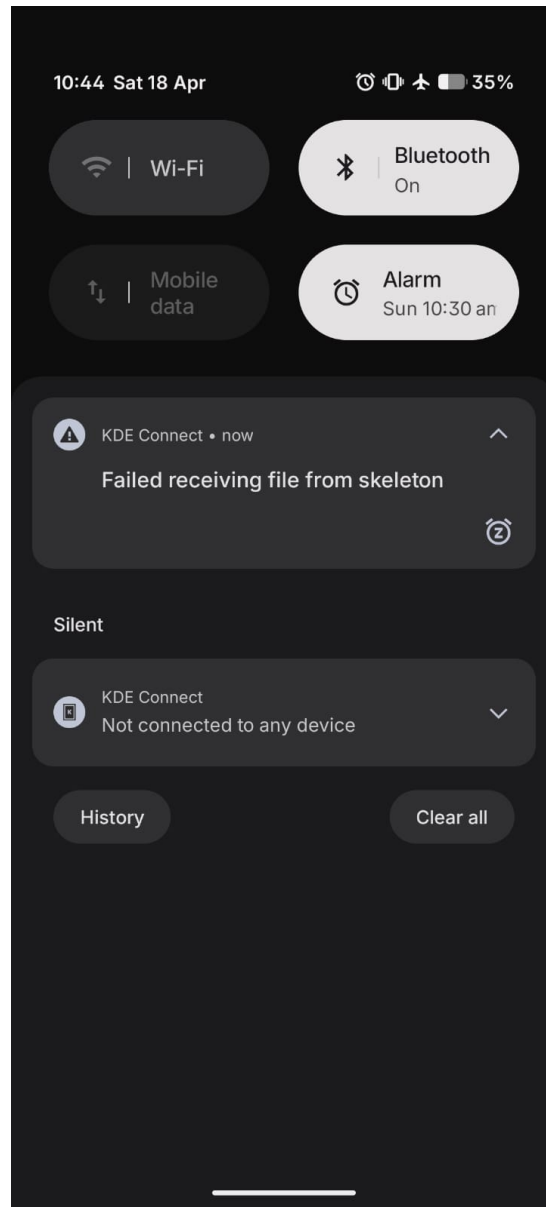


Figure 10: A1 antipattern: “Failed to receive” notification after a transient Wi-Fi drop; the partial file is already gone.

9.6 A2. The outgoing packet queue is lost when the last link disconnects

Reading `Device.kt` we noticed that each `Device` owns a Kotlin `Channel<NetworkPacketWithCallback>(Channel.UNLIMITED)` consumed by a dedicated coroutine. When the last `BaseLink` is removed (i.e. the device becomes unreachable), the coroutine is cancelled. Everything still in the channel buffer is silently discarded, and nothing is persisted to disk.

Evidence – `Device.kt`:

Listing 12: `Device.kt`: in-memory send queue cancelled on disconnect

```

1 private val sendChannel = Channel<NetworkPacketWithCallback>(Channel.
    UNLIMITED)
2 private var sendCoroutine: Job? = null
3 ...
4 fun removeLink(link: BaseLink) {
5     ...

```

```

6      if (links.isEmpty()) {
7          reloadPluginsFromSettings()
8          synchronized(sendChannel) {
9              sendCoroutine?.cancel(
10                 CancellationException("Device disconnected")) // <--
11                 queue dropped
12                 sendCoroutine = null
13             }
14         }
15     }

```

The method `sendPacketBlocking` makes the loss even more explicit when no link succeeds:

Listing 13: Device.kt: “Packet lost!”

```

1  if (!success) {
2      Log.e("KDE/sendPacket",
3          "No device link (of ${links.size} available) could send the
4          packet. "
5          + "Packet ${np.type} to ${deviceInfo.name} lost!")
6  }
7  return success

```

This is a problem because plugins that call `device.sendPacket(np)` (battery updates, clipboard, SMS events, volume, MPRIS state, telephony events) assume a fire-and-forget contract, but during a brief disconnect the caller receives no failure signal and the message is gone. The behaviour also compounds with the antipattern we describe next (A3).

Reproduction. We reproduced the packet loss with the following steps:

1. We paired phone and desktop.
2. On the desktop we launched `adb logcat -s KDE/sendPacket` (any Logcat viewer filtered by “Packet...lost” also works).
3. On the phone we toggled Wi-Fi off briefly and back on.
4. During the off window we copied text on the phone to trigger a clipboard packet.
5. Back in logcat, we saw one or more “Packet ...lost!” lines produced by `sendPacketBlocking`.

Proposed fix. Flush the `sendChannel` to a disk-backed queue (for example, a small Room table keyed by device + monotonic id) on `removeLink`, and re-enqueue on `addLink`. Plugins that care about ordering (clipboard, SMS) already include timestamps in the payload, so deduplication on the receiver side is easy.

9.7 A3. The outbox pattern is only implemented for URLs, not for text or files

When we looked at how A2’s loss interacts with the sharing flow, we found that the outbox in G1 is triggered only when `doesIntentContainUrl(intent)` returns true. For plain-text or file shares, `ShareActivity` does not even show unreachable devices in the picker (so the user cannot queue), and the plain-text branch of `SharePlugin.share()` goes straight to `device.sendPacket(np)`, which makes it subject to A2.

Evidence – `ShareActivity.java` and `SharePlugin.java`:

Listing 14: ShareActivity.java: offline devices hidden unless intent is a URL

```

1  for (Device d : devices) {
2      // Show the paired devices only if they are unreachable
3      // AND the shared intent has a URL

```

```

4     if (d.isPaired() && (intentHasUrl || d.isReachable())) {
5         devicesList.add(d);
6         if (!d.isReachable()) {
7             items.add(new UnreachableDeviceItem(d, ...)); // URL-only
8         } else {
9             items.add(new DeviceItem(d, ...));
10        }
11    }
12 }

```

Listing 15: SharePlugin.java: plain-text share relies on fire-and-forget

```

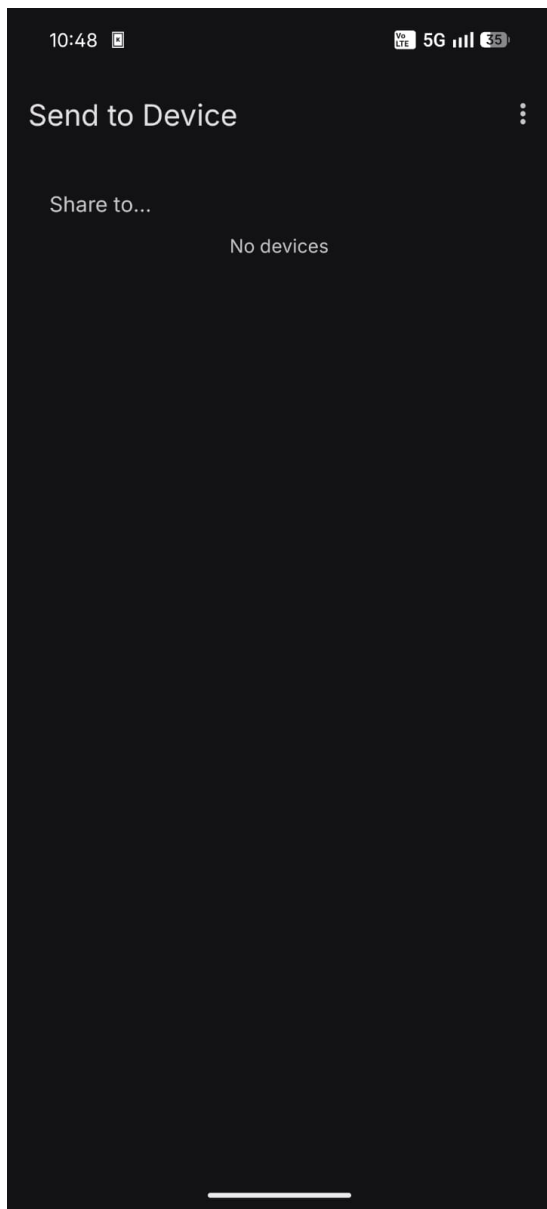
1 NetworkPacket np = new NetworkPacket(SharePlugin.
    PACKET_TYPE_SHARE_REQUEST);
2 np.set(isUrl ? "url" : "text", text);
3 device.sendPacket(np); // <-- dropped if device just disconnected (see
    A2)

```

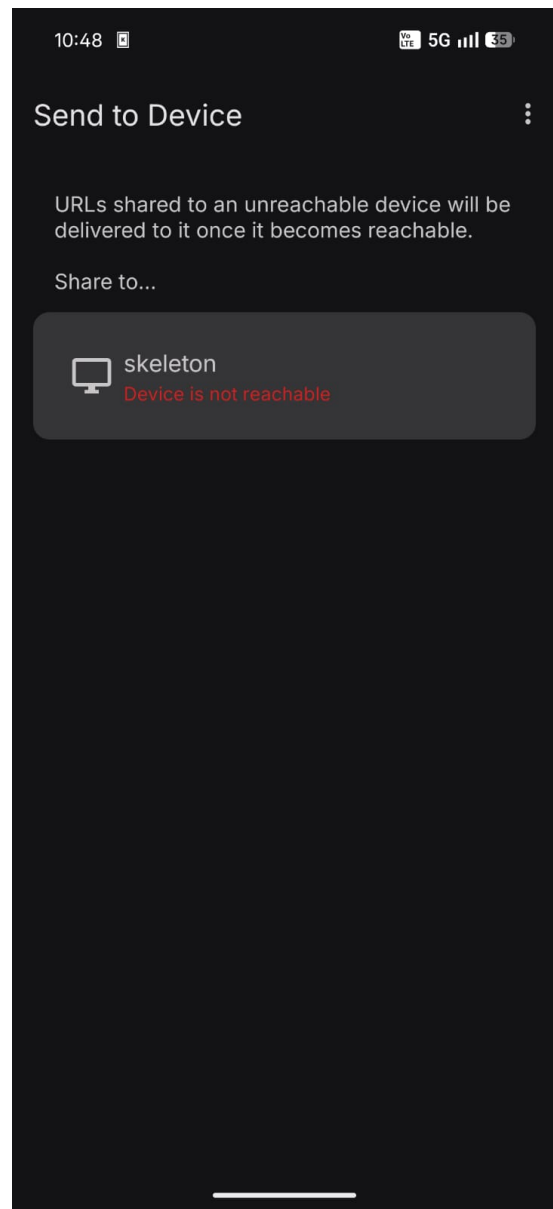
Reproduction. To make the asymmetry visible we ran two shares back to back (Figure 11):

1. On the phone, we disabled Wi-Fi so the paired desktop became unreachable.
2. We opened a browser, selected a passage of plain text (not a link), tapped *Share* and picked KDE Connect. The picker either omitted the desktop entirely or greyed it out; the text was never queued.
3. We repeated the step using an http(s) URL: the desktop was listed (even offline) and the picker toast read “Will be delivered when device comes online,” confirming that the outbox exists but is URL-only.

Proposed fix. Generalise `storeUrlForFutureDelivery` to `storePendingShare(device, intentBundle)` with a bounded on-disk queue, and drain it in `SharePlugin.deliverPreviously-SentIntents()`. This single change would close the gap exposed by A2 and A3 for share-related packets.



(a) Sharing *plain text* to an offline paired device: the device is absent / not selectable, no outbox is created.



(b) Contrast: sharing an *http(s) URL* to the *same* offline device — it is now listed and the toast announces deferred delivery.

Figure 11: A3 antipattern vs. its positive counterpart (G1). Same peer, same offline state, only the payload type changes.

9.8 ECn summary

Overall, we consider that KDE Connect handles reconnection very well (G1–G4) but handles pending work poorly (A1–A3). In our reading, the root cause is a single architectural gap: there is no generic, persistent outbox for outgoing `NetworkPackets` — only a best-effort in-memory `Channel` and one hard-coded special case for URLs.

10 Caching Strategies

The rubric asks us to identify at least three caching structures ($X = \text{number of members} = 3$). Grepping the codebase for cache-like patterns, we identified three with clearly different trade-offs

plus one bonus in-memory structure that also acts as a cache. Their sizes and placements are summarised in Table 2.

10.1 C1. DiskLruCache for album art

Reading `plugins/mpris/AlbumArtCache.kt` we saw that the MPRIS plugin receives album-art URLs from the desktop (`http(s)`, `file://` or `kdeconnect://`) and caches the decoded bytes so that scrubbing tracks or reopening the player does not re-transfer images. The cache is Jake Wharton's `DiskLruCache`, stored under `context.cacheDir/album_art` (`app-private`).

Evidence – `AlbumArtCache.kt`:

Listing 16: `AlbumArtCache.kt`: disk cache initialization

```
1 fun initializeDiskCache(context: Context) {
2     if (this::diskCache.isInitialized) return
3     val cacheDir = File(context.cacheDir, "album_art")
4     // Initialize the disk cache with a limit of 5 MB storage
5     // (fits ~830 images, taking Spotify as reference)
6     diskCache = DiskLruCache.open(
7         cacheDir, BuildConfig.VERSION_CODE, 1, 1000 * 1000 * 5.toLong())
8     ...
9 }
10
11 private fun urlToDiskCacheKey(url: String): String =
12     MessageDigest.getInstance("MD5").digest(url.toByteArray())
13     .joinToString(separator = "") { String.format("%02x", it) }
```

We believe the developers chose a bounded disk LRU because album covers are large (tens to hundreds of kB), re-downloading them on every reconnect would waste bandwidth on metered networks, and storing them forever would grow unbounded. The 5 MB cap is conservative and, according to the in-code comment, fits around 830 covers measured against Spotify.

10.2 C2. In-memory LruCache for already-decoded bitmaps

On top of the disk cache, `AlbumArtCache` keeps a small in-memory cache of already-decoded `Bitmaps`, and it also remembers *failed* fetches so the app does not keep retrying URLs that will never resolve.

Evidence – `AlbumArtCache.kt`:

Listing 17: `AlbumArtCache.kt`: memory cache + failure memoization

```
1 // Holds at most 10 entries (to prevent too much memory usage)
2 // Also remembers failure to fetch urls.
3 private val memoryCache = LruCache<String, MemoryCacheItem>(10)
4
5 private class MemoryCacheItem(
6     var failedFetch: Boolean = false,
7     var albumArt: Bitmap? = null)
```

We read this layering as a deliberate choice: decoding a PNG or JPEG off disk is expensive enough that the last-used handful of bitmaps should stay hot in RAM, but 830 bitmaps in memory would cost hundreds of MB. So RAM is capped at 10 entries and the disk layer absorbs the rest.

10.3 C3. SharedPreferences as a configuration and trust cache

KDE Connect uses `SharedPreferences` (backed by an XML file in the app’s private data directory) as a lightweight key–value cache for data that is read on every connection attempt and that is expensive to recompute. We list two representative examples below.

Evidence – `TrustedDevices.kt`:

Listing 18: `TrustedDevices.kt`: “is this peer trusted?” cache

```
1 fun isTrustedDevice(context: Context, deviceId: String): Boolean {
2     val preferences = context.getSharedPreferences("trusted_devices",
3         MODE_PRIVATE)
4     return preferences.getBoolean(deviceId, false)
5 }
6 fun getDeviceCertificate(context: Context, deviceId: String):
7     Certificate {
8     val devicePreferences = context.getSharedPreferences(deviceId,
9         MODE_PRIVATE)
10    val certificateBytes = Base64.decode(
11        devicePreferences.getString("certificate", ""), 0)
12    return parseCertificate(certificateBytes)
13 }
```

This file is consulted on every UDP identity packet received in `LanLinkProvider.unserializeReceivedIdentityPacket()` to decide whether to proceed with the TLS handshake. Without this cache, each handshake would have to re-read the certificate from a DER file or, worse, force the user to re-pair every time.

10.4 C4. ConcurrentHashMap-based in-memory caches

In addition to the three caches above, we found two in-memory structures that act as de-facto caches and are worth mentioning:

- `KdeConnect.devices` of type `ConcurrentHashMap<String, Device>`: caches the `Device` objects so every UI lookup and every incoming packet lands in $O(1)$; this avoids rebuilding devices from `SharedPreferences` on every request.
- `LanLinkProvider.lastConnectionTimeByDeviceId` and `lastConnectionTimeByIp`, both `ConcurrentHashMaps` that cache timestamps for the rate-limiting logic (see G3), bounded at `MAX_RATE_LIMIT_ENTRIES = 255` and self-pruned.

10.5 Average amount of cache storage

Table 2 summarises what we measured and derived from the code.

Structure	Where / what	Cap	Typical
<code>DiskLruCache</code>	<code>cacheDir/album_art</code> , cover images	5 MB	1–3 MB
<code>LruCache<Bitmap></code>	decoded album covers in RAM	10 entries	1–5 MB RAM
<code>SharedPreferences</code>	<code>trusted_devices.xml</code> plus one file per paired device (certificate, plugin settings)	none set	2–8 kB per paired device
<code>ConcurrentHashMap</code>	<code>KdeConnect.devices</code> and rate-limit maps	255 entries (rate-limit)	a few kB total

Table 2: Caching structures, upper bounds, and typical footprint we observed on the test device.

For a normal home or office setup (two or three paired devices, casual MPRIS use) total cache

storage sits comfortably under 10MB on disk and a few MB in RAM, which we consider proportionate for an app that has to be always-on in a foreground service.

11 Memory Management Strategies

The rubric asks for at least three memory-management mechanisms ($X = 3$ members). We identified four, each associated with specific functionalities. For every one we analyse the two questions the rubric asks: should the data be persistent or transient, and should it live in a private or a public space.

11.1 M1. SharedPreferences (persistent, private)

Associated functionality. Trust bookkeeping, per-device plugin settings, theme preference, the “last seen” clipboard timestamp, the Share plugin URL outbox (G1), and the “disable persistent notification” toggle.

Evidence – `TrustedDevices.kt`:

Listing 19: `TrustedDevices.kt`: per-device settings file

```
1 fun getDeviceSettings(context: Context, deviceId: String):  
    SharedPreferences {  
2     return context.getSharedPreferences(deviceId, MODE_PRIVATE)  
3 }  
4  
5 fun addTrustedDevice(context: Context, deviceId: String) {  
6     val preferences = context.getSharedPreferences("trusted_devices",  
        MODE_PRIVATE)  
7     preferences.edit { putBoolean(deviceId, true) }  
8 }
```

Persistent or transient. We think this data should be, and is, persistent: the TLS certificate of every paired peer has to survive reboots, and any other choice would force re-pairing on every cold start. **Private or public.** All calls use `MODE_PRIVATE`, so the files live inside the app’s sandbox under `/data/data/org.kde.kdeconnect_tp/shared_prefs/`. We consider this the right choice: certificates and device identifiers are sensitive, and KDE Connect has no need to expose them to other apps.

11.2 M2. SQLite via SQLiteOpenHelper (persistent, private)

Associated functionality. Per-application notification filtering: which installed apps are allowed to forward notifications to the desktop, plus per-app privacy bits (“block contents”, “block images”). We think `SharedPreferences` would not have been a good fit here because the working set is a few hundred rows queried by `packageName` and combined with a privacy bitmask.

Evidence – `plugins/notifications/AppDatabase.java`:

Listing 20: `AppDatabase.java`: schema of the notifications table

```
1 private static final String DATABASE_CREATE_ENABLED = "CREATE TABLE "  
2     + TABLE_ENABLED + "(" + KEY_PACKAGE_NAME + " TEXT PRIMARY KEY NOT  
    NULL, "  
3     + KEY_IS_ENABLED + " INTEGER NOT NULL ); ";  
4 private static final String DATABASE_CREATE_PRIVACY_OPTS = "CREATE TABLE  
    "  
5     + TABLE_PRIVACY + "(" + KEY_PACKAGE_NAME + " TEXT PRIMARY KEY NOT  
    NULL, "
```



```

6      + KEY_PRIVACY_OPTIONS + " INTEGER NOT NULL); ";
7
8  void setEnabled(String packageName, boolean isEnabled) {
9      SQLiteDatabase db = ourHelper.getWritableDatabase();
10     ...
11     db.update(TABLE_ENABLED, cv, KEY_PACKAGE_NAME + "=?",
12               new String[]{packageName});
13 }

```

Persistent or transient. Persistent: users do not want to rebuild their notification-filtering block-list every reboot. **Private or public.** Private, since the database lives under `/data/data/.../-databases/`. We consider this the right choice: notification-filtering choices reveal which apps the user has installed, which is personal data that should not leak.

11.3 M3. File storage (mix of private cache and user-visible public)

Associated functionality. Two very different use cases share this mechanism:

- The album-art cache ends up in `context.cacheDir/album_art/`, which is app-private and semantically transient (Android may purge cache directories under memory pressure).
- Received files from the Share plugin end up either in the user-chosen destination via the Storage Access Framework, or in the default Downloads directory for “open” shares.

Evidence – `CompositeReceiveFileJob.java`:

Listing 21: `CompositeReceiveFileJob.java`: choice of destination

```

1  if (open || !ShareSettingsFragment.isCustomDestinationEnabled(ctx)) {
2      final String defaultPath =
3          ShareSettingsFragment.getDefaultDestinationDirectory().
4              getAbsolutePath();
5      filenameToUse = FilesHelper
6          .findValidNonExistingFileNameForFile(defaultPath, filenameToUse)
7          ;
8      destinationFolderDocument = DocumentFile.fromFile(new File(
9          defaultPath));
10 } else {
11     destinationFolderDocument = ShareSettingsFragment
12         .getDestinationDirectory(getDevice().getContext());
13 }
14 ...
15 if (!ShareSettingsFragment.isCustomDestinationEnabled(ctx)) {
16     DownloadManager manager = ContextCompat.getSystemService(ctx,
17         DownloadManager.class);
18     manager.addCompletedDownload(...); // indexes the file publicly
19 } else {
20     MediaStoreHelper.indexFile(ctx, fileDocument.getUri());
21 }

```

Persistent or transient. Album art is transient: it can always be regenerated, and the cache directory is the right semantic choice because Android will reclaim it if space is tight and the app tolerates misses. Received files are persistent by intent: the user expects them to stick around and be visible in the Gallery or Downloads. **Private or public.** Album art is private (sandbox). Received files are exposed to the rest of the system: they are registered with `DownloadManager` or `MediaStore` so the Gallery and file manager can find them. We consider this correct: users explicitly transfer files to use them in other apps, while credentials, certificates, and tokens are never stored in this location.

11.4 M4. In-memory structures with explicit lifecycle (transient, in-process)

Associated functionality. The device list, plugin instances, listener lists (device-changed, plugin-changed, pairing callbacks), and the per-device outgoing packet channel.

Evidence – Device.kt:

Listing 22: Device.kt: thread-safe in-memory collections

```
1 private val links = CopyOnWriteArrayList<BaseLink>()
2 val loadedPlugins: ConcurrentMap<String, Plugin> = ConcurrentHashMap()
3 val pluginsWithoutPermissions: ConcurrentMap<String, Plugin> =
    ConcurrentHashMap()
4 private val pairingCallbacks = CopyOnWriteArrayList<PairingCallback>()
5 private val pluginsChangedListeners = CopyOnWriteArrayList<
    PluginsChangedListener>()
6 private val sendChannel = Channel<NetworkPacketWithCallback>(Channel.
    UNLIMITED)
```

Persistent or transient. Transient: all of this is rebuilt from `SharedPreferences` at process start by `KdeConnect.loadRememberedDevicesFromSettings()`. Keeping it in RAM is essential because every incoming packet has to be routed in microseconds. **Private or public.** In-process only; nothing is exposed through any `ContentProvider` or binder surface, so the app itself is the only consumer.

Lifecycle hygiene. We checked the listener registrations and found that they are explicitly added in `onCreate` or `onStart` and removed in `onDestroy` or `onDestroyView`, which avoids the usual leak of the enclosing fragment:

Evidence – DeviceFragment.kt:

Listing 23: DeviceFragment.kt: add/remove listener

```
1 device?.addPluginsChangedListener(pluginsChangedListener)
2 // ... in onDestroyView:
3 device?.removePluginsChangedListener(pluginsChangedListener)
```

The `CopyOnWriteArrayList` was, in our reading, chosen so iteration from the networking thread does not need a lock while the UI thread adds and removes listeners.

Cleanup of dangling devices. When we read `KdeConnect.scheduleForDeletion()` we found a long comment in which the author acknowledges that `Device` can be leaked through the UI if a view keeps a strong reference, and even suggests using the debugger with `Runtime.getRuntime().gc()` to check for leaks. We found this unusually honest and read it as evidence that the team actively fights retention bugs.

Evidence – KdeConnect.kt:

Listing 24: KdeConnect.kt: device GC reasoning

```
1 fun scheduleForDeletion(device: Device) {
2     // Note: By this point the 'devices' map should be the only
        reference
3     // to 'device', so it should get garbage collected after removing it
4     // here. However, it's easy to leak references to Device from views,
5     // preventing them from being freed...
6     ThreadHelper.execute {
7         try { Thread.sleep(1000) } catch (_, InterruptedException) { }
8         if (device.isReachable || device.isPaired) return@execute
9         device.removePairingCallback(devicePairingCallback)
```

```

10         devices.remove(device.deviceId)
11     }
12 }

```

11.5 Summary of placement decisions

Table 3 consolidates the four mechanisms we discussed, answering the two questions the rubric asks for every piece of data.

Mechanism	Functionality	Lifetime	Visibility
Shared-Preferences	trust, per-device settings, URL outbox, theme	persistent	private
SQLite (App-Database)	notification filtering and privacy bits	persistent	private
cacheDir/-album_art	MPRIS album-art cache	transient (system-evictable)	private
Downloads, Media-Store	received files (Share plugin)	persistent	public (intentional)
Concurrent-HashMap, Copy-OnWriteArray-List, Channel	devices, plugins, listeners, outgoing queue	transient (in-memory)	(in- in-process only)

Table 3: Memory-management mechanisms and their placement.

Overall, we read the placement as deliberate: anything sensitive (certificates, trust data, notification filter) is private and persistent; anything the user wants to reuse outside the app (received files) is public and persistent; anything that can be rebuilt (album art, in-memory device list) uses a transient or evictable location. The one weak point, already discussed in §9, is that outgoing packets live only in the in-memory **Channel** and do not get demoted to **SharedPreferences** or to a Room queue when the link dies.

12 Threading / Concurrency Strategies

KDE Connect is I/O-bound (sockets, TLS handshakes, file payloads) and it also reacts to system broadcasts, so we expected to find more than one concurrency primitive. Reading through the codebase we identified five distinct strategies, each associated with specific functionality.

12.1 T1. Shared ExecutorService through ThreadHelper

We found that a single `Executors.newCachedThreadPool()` is exposed app-wide as `ThreadHelper.execute{}`. It is used as the default “do this off the main thread” primitive across the code: UDP listeners, TCP accept loops, TLS handshake callbacks, scheduled device GC, etc.

Evidence – `helpers/ThreadHelper.kt`:

Listing 25: `ThreadHelper.kt`: the shared cached pool

```

1 object ThreadHelper {
2     private val executor: ExecutorService = Executors.
3         newCachedThreadPool()
4     @JvmStatic fun execute(command: Runnable) = executor.execute(command)
5     fun assertMainThread() { ... }

```

```

5     fun assertNotMainThread() { ... }
6 }

```

Associated functionality. Every long-running network operation in `LanLinkProvider`, including the accept loops for UDP and TCP and the TLS handshake callback, is submitted to this pool:

Evidence – `LanLinkProvider.java`:

Listing 26: `LanLinkProvider.java`: accept loops run on `ThreadHelper`

```

1 ThreadHelper.execute(() -> {
2     while (listening) {
3         Socket socket = tcpServer.accept();
4         configureSocket(socket);
5         ThreadHelper.execute(() -> {                               // nested: one task per
6             try { tcpPacketReceived(socket); }                      peer
7             catch (IOException e) { ... }
8         });
9     }
10 });

```

In our reading, a cached pool matches the expected load (bursts of short-lived handshake tasks) without pre-allocating threads, and it reclaims idle threads after 60s.

12.2 T2. Kotlin coroutines for the per-device send queue and background work

The per-device outgoing packet queue we already discussed in §9 and §11 is implemented with coroutines and a `Channel` consumed on `Dispatchers.IO`.

Evidence – `Device.kt`:

Listing 27: `Device.kt`: send-packet coroutine

```

1 private val sendChannel = Channel<NetworkPacketWithCallback>(Channel.
   UNLIMITED)
2 private var sendCoroutine: Job? = null
3
4 fun addLink(link: BaseLink) {
5     synchronized(sendChannel) {
6         if (sendCoroutine == null) {
7             sendCoroutine = CoroutineScope(Dispatchers.IO).launch {
8                 for ((np, callback) in sendChannel) {
9                     sendPacketBlocking(np, callback)
10                }
11            }
12        }
13    }
14    ...
15 }
16
17 fun launchBackgroundReloadPluginsFromSettings() {
18     CoroutineScope(Dispatchers.IO).launch {
19         reloadPluginsFromSettings()
20     }
21 }

```

Coroutines are also used by `AlbumArtCache` to run HTTP fetches off the main thread under a `SupervisorJob`, so a single failing fetch does not cancel the others:

Evidence – `AlbumArtCache.kt`:

Listing 28: `AlbumArtCache.kt`: supervised IO scope

```
1 private val cacheScope = CoroutineScope(Dispatchers.IO + SupervisorJob())
2 ...
3 cacheScope.launch { fetchURL(url, payload, cacheItem) }
```

Associated functionality. Per-device outgoing packet queue, plugin reload, and album-art network fetches.

12.3 T3. BackgroundJobHandler: a fixed pool with Future management

File-transfer jobs (`CompositeReceiveFileJob` and `CompositeUploadFileJob`) do not use coroutines. Instead, they use a custom `ThreadPoolExecutor` that tracks `Future` objects, so jobs can be cancelled mid-transfer and uncaught exceptions can be routed to the callback on the main thread.

Evidence – `async/BackgroundJobHandler.kt`:

Listing 29: `BackgroundJobHandler.kt`: fixed pool with uncaught-exception routing

```
1 class BackgroundJobHandler {
2     private inner class MyThreadPoolExecutor(...) : ThreadPoolExecutor
3         (...) {
4         override fun afterExecute(runnable: Runnable, throwable:
5             Throwable?) {
6             super.afterExecute(runnable, throwable)
7             ...
8             if (t != null) this@BackgroundJobHandler.
9                 handleUncaughtException(future, t)
10        }
11    }
12
13    fun runJob(bgJob: BackgroundJob<*, *>) {
14        bgJob.setBackgroundJobHandler(this)
15        val future: Future<*> = threadPoolExecutor.submit(bgJob)
16        jobMap.put(bgJob, future)
17    }
18
19    fun runOnUiThread(runnable: Runnable) { handler.post(runnable) }
20
21    companion object {
22        fun newFixedThreadPoolBackgroundJobHandler(numThreads: Int) =
23            BackgroundJobHandler(numThreads, numThreads, 0L,
24                TimeUnit.MILLISECONDS, LinkedBlockingQueue())
25    }
26 }
```

We checked the caller and found that `SharePlugin` instantiates this handler with 5 threads, which is the effective ceiling on parallel file transfers to or from one device.

Associated functionality. File send and receive, and pairing-related bulk work.

12.4 T4. Dedicated Thread + Loopers for the SMS content observer

The SMS plugin observes `content://mms-sms/...` changes through a `ContentObserver`, which requires a `Looper` to post callbacks onto. Instead of blocking the main looper, `SMSTHelper` spins up a single background thread with its own looper.

Evidence – `helpers/SMSTHelper.kt`:

Listing 30: `SMSTHelper.kt`: the singleton looper thread

```
1 class MessageLooper private constructor() : Thread() {
2     init { setName("MessageHelperLooper") }
3
4     override fun run() {
5         looperReadyLock.lock()
6         try {
7             Looper.prepare()
8             looper = Looper.myLooper()
9             looperReady.signalAll()
10        } finally { looperReadyLock.unlock() }
11        Looper.loop()
12    }
13
14    companion object {
15        @JvmStatic
16        fun getLooper(): Looper? {
17            if (singleton == null) {
18                looperReadyLock.lock()
19                try {
20                    singleton = MessageLooper().apply { start() }
21                    while (looper == null) { looperReady.await() }
22                } ...
23            }
24            return looper
25        }
26    }
27 }
```

The looper is used by `SMSTPlugin` to drive the observer:

Evidence – `SMSTPlugin.kt`:

Listing 31: `SMSTPlugin.kt`: observer runs on the dedicated looper

```
1 private val messageObserver: ContentObserver =
2     MessageContentObserver(Handler(getLooper()!!))
```

Associated functionality. Change-stream of SMS and MMS messages for the “forward new SMS to desktop” feature. In our view a dedicated looper is the right choice because SMS queries can be slow (cursor over the system provider) and must not block the UI.

12.5 T5. Lock-free and lock-biased data structures for the UI and worker boundary

Throughout the codebase we noticed a consistent pattern for sharing state between worker threads and the UI. Read-heavy, write-rare lists use `CopyOnWriteArrayList` (for example `links`, `pluginsChangedListeners`, `pairingCallbacks`, and `registeredPlugins` in `AlbumArtCache`), so iteration from worker threads does not need external synchronization. Maps that are mutated from multiple threads use `ConcurrentHashMap` (for example `KdeConnect.devices`, `loadedPlugins`,

and the rate-limit maps). The send queue uses `Channel.UNLIMITED`, which provides a lock-free producer/consumer. Fine-grained locking appears where really needed: a `ReentrantLock` in `SMSPlugin.mostRecentTimestampLock` guards the message-timestamp watermark, and a `synchronized(lock)` block in `CompositeReceiveFileJob` protects the per-packet list.

Associated functionality. Cross-thread notification of state changes (device list, plugin list, pairing), the SMS watermark, and the file-receive packet list.

12.6 Main-thread marshalling

Workers marshal results back to the UI via `Handler.post` (`BackgroundJobHandler.runOnUiThread`) and `Activity.runOnUiThread` (`DeviceFragment`), as the Android framework requires:

Evidence – `DeviceFragment.kt`:

Listing 32: `DeviceFragment.kt`: UI thread marshalling

```
1 private val pluginsChangedListener = PluginsChangedListener {  
2     mActivity?.runOnUiThread { refreshUI() }  
3 }
```

Debug builds enable `StrictMode` in `KdeConnect.onCreate()` with detectors such as `detectUnbufferedIo`, `detectLeakedClosableObjects`, and `detectIncorrectContextUse`, so threading and I/O mistakes surface in logcat during development.

12.7 Threading summary

KDE Connect does not rely on a single concurrency model. It mixes a cached pool for short bursts (T1), coroutines where cancellation and structured concurrency matter (T2), a fixed-size pool with `Futures` for long cancellable file jobs (T3), a dedicated loop for system `ContentObserver` callbacks (T4), and carefully chosen lock-free and lock-biased containers to bridge workers and UI (T5). The trade-off is code variety, but we believe the model matches the workload, and in all five cases we were able to map the strategy to a concrete functionality in the app.

Part IV

Part 4

13 Methodology and device under test

This Part 4 is the technical audit component of the report. We identify and document concrete technical issues in KDE Connect Android using Android Studio Profiler as the primary instrument and the architectural map produced in Parts 1–3 as the diagnostic baseline. The issues reported here include a real memory leak in `DeviceFragment` (which we patched and submitted upstream as KDE Bugzilla 520075 and merge request !649), a refinement of anti-pattern A1 from Part 3 (partial files left on disk after a transfer failure), three janky fragment transitions during cold start, and an unbounded cached thread pool documented as anti-pattern A4. For each issue we record the measurement that surfaced it, the root cause, and a concrete mitigation.

Part 4 reuses the architectural map produced in Parts 1–3 and grounds it in measurements taken with Android Studio Profiler. We chose three scenarios that together exercise the dominant code paths identified earlier: the cold start path (which loads `KdeConnect`, the plugin registry and the network providers), the share path (which exercises T3 – the fixed pool backing file transfers –

and the network stack), and a configuration change (which stresses fragment and view recreation). The matching scenario names below are S1, S2 and S3.

Device under test. Xiaomi 21061119AL (POCO M6 Pro), Android 12 (API 31), 8-core CPU. The device does not expose the on-device power monitor (ODPM, available from Pixel 6 onwards), so power readings throughout Part 4 come from the **BatteryManager** estimate that Android Studio Profiler displays. Where the value is qualitative we keep the Profiler classification (*Light* / *Moderate* / *Heavy*).

Build. Debug, signed with the project’s debug keystore. The Profiler warned that the process recommends being marked **profileable**; we proceeded because the difference between debug and profileable for the metrics in scope (CPU%, heap, frame timings) is small enough to keep the qualitative findings valid.

Captures per scenario.

- S1: a single System Trace started in **Process Start** mode (cold start visibility from millisecond 0).
- S2: three heap dumps (idle, mid-transfer, post-failure) plus one post-fix heap dump after applying the contributed patch.
- S3: live telemetry over ten rotations plus two heap dumps (before and after the cycle, post-GC).

Leak detection libraries. The de-facto Android library for leak detection in app code is Square’s LeakCanary (<https://square.github.io/leakcanary/>). KDE Connect does not ship it because the upstream project is distributed through F-Droid, which requires reproducible builds with no proprietary or instrumented runtime dependencies. Our equivalent workflow was: take heap dumps in the Profiler at well-defined points in each scenario, filter by the suspected class, and walk the *References* tab to find the GC root that retains the instance. We used this exact workflow to find, file and patch the leak in **DeviceFragment** discussed in S2.

14 Scenario S1 – Cold start and UI baseline

14.1 Scenario definition

S1 measures the time from a fully stopped process to a useful UI, and the baseline behaviour after that. The flow is:

1. Force-stop the application from system settings (true cold start, not warm start).
2. Launch from the launcher icon and wait for the device list.
3. Open the navigation drawer.
4. Tap the paired device (**JunF3rm4t21**) to enter **DeviceFragment**.
5. Press back to return to the device list.
6. Open the drawer again and tap **Settings**.
7. Press back.

The trace lasted 26.4 s.

14.2 CPU consumption and threading

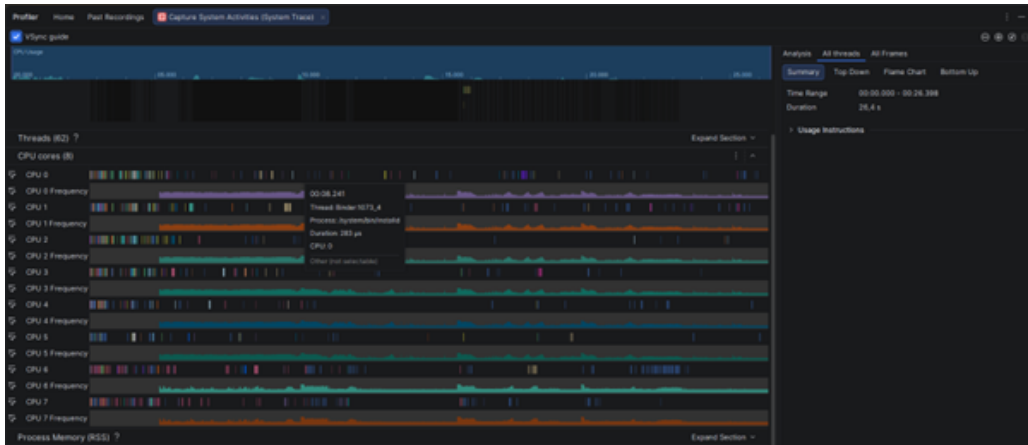


Figure 12: S1.1 – Work distribution across the eight CPU cores. Activity concentrates in the first three seconds (cold start) and then settles into an idle pattern.

The main thread spent most of the trace in the **Sleeping** state, with brief bursts of activity for each user interaction. Table 4 summarises the state breakdown reported by the Profiler.

Table 4: S1 – main thread state distribution over the 26.4 s trace.

State	Duration	%	Occurrences
Sleeping	19.64 s	74.39%	366
No thread available	5.00 s	18.94%	1
Running	1.05 s	3.97%	1095
Unknown	611 ms	2.32%	504
Runnable	94 ms	0.36%	916
Waiting	6 ms	0.02%	15

The five longest events recorded on the main thread are listed in Table 5. `bindApplication` and `activityStart` are framework-driven cold-start work, not application code; the high wall-clock vs CPU ratio on `Choreographer.doFrame` (530 ms wall, 1.45 ms CPU) indicates the main thread was waiting for a VSYNC signal rather than doing work.

Table 5: S1 – top main-thread events by wall duration.

Event	Wall	CPU
<code>Choreographer.doFrame</code>	530.7 ms	1.45 ms
<code>traversal</code>	528.5 ms	211.6 ms
<code>bindApplication</code>	476.6 ms	83.2 ms
<code>activityStart</code>	339.5 ms	9.81 ms
<code>performTraversal</code>	299.4 ms	177.7 ms

The thread count peaked at 62 across the process. No blocking work on the main thread other than the framework cold-start path was observed.

14.3 Memory management

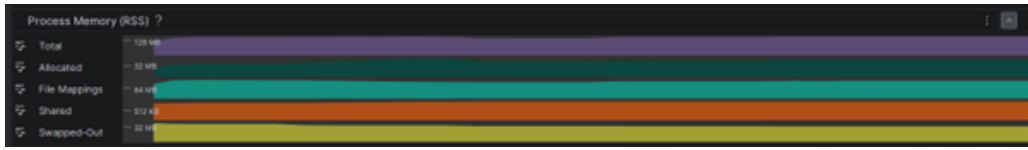


Figure 13: S1.2 – RSS over the trace. Lines are flat after the initial ramp.

Table 6: S1 – memory breakdown at steady state.

Region	Size
Total RSS	128 MB
Allocated (Java/Kotlin)	32 MB
File mappings	64 MB
Shared	512 KB
Swapped-out (zRAM)	32 MB

The 64 MB of file mappings is dominated by the native security libraries (Bouncy Castle and the embedded SSHD); the 32 MB of zRAM is normal Android behaviour when the kernel compresses cold pages to free physical RAM. Java heap stayed near 32 MB throughout. No leaks were observed in this scenario; GC events were sparse, consistent with the low allocation rate of an idle UI.

14.4 GPU rendering analysis

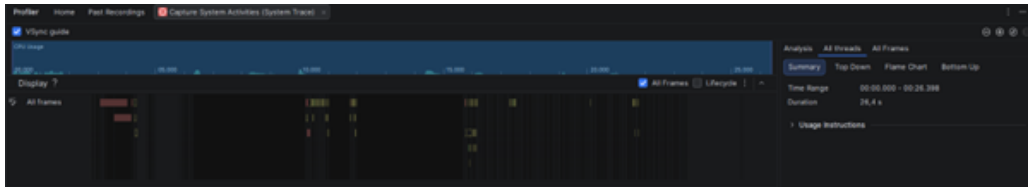


Figure 14: S1.3 – Frame timeline. The red bars (jank) cluster around cold start and fragment transitions.

The Profiler reported 131 frames over the trace, of which 7 were classified as jank (5.34%). Table 7 breaks the seven events down. The longest frame (1.06 s) corresponds to the very first frame after `bindApplication`; the remaining six are split between two composition stalls and four fragment transitions where the `DeviceFragment` plugin cards were being inflated.

Table 7: S1 – per-frame breakdown of the seven janky frames.

Frame	Total	App	GPU	Comp.	Likely cause
2080386	1.06 s	197.9 ms	—	5.75 ms	Cold start, first frame
2080444	675.5 ms	12.04 ms	—	20.07 ms	Composition stall
2080470	52.7 ms	7.5 ms	1.34 ms	25.75 ms	Slow composition
2080582	79.5 ms	71.79 ms	—	17.39 ms	Fragment transition
2080585	144.99 ms	72.69 ms	1.26 ms	12.25 ms	Heavy fragment transition
2080684	21.5 ms	7.29 ms	11.9 ms	12.31 ms	Borderline
2080722	60.9 ms	53.75 ms	3.94 ms	16.5 ms	Fragment transition

14.5 Overdraw

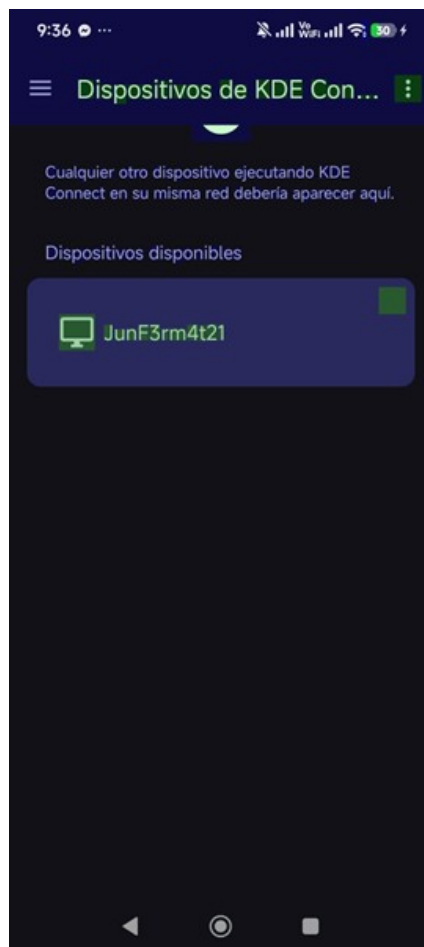


Figure 15: S1.4 – Overdraw on the device-list screen. Blue (1x) dominates with localised green (2x) on list items.

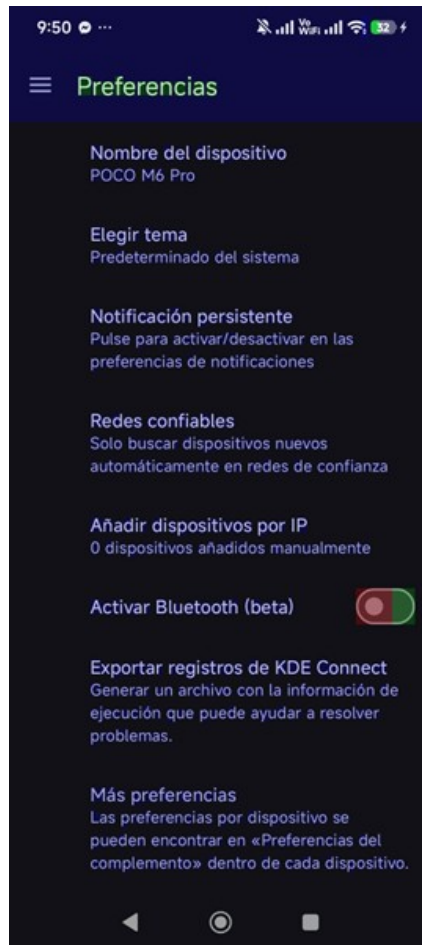


Figure 16: S1.5 – Overdraw on the Settings screen. A small red (3x+) region appears around one container.

The main screen draws at 1x to 2x throughout, which is the expected range. The Settings screen has a single red region that suggests a redundant background layer in one container; this is a localised issue and does not show up in CPU or GPU times.

14.6 Power consumption



Figure 17: S1.6 – Battery panel during S1. A brief ramp during cold start, then flat.

Classification: *Light*. The reading is the `BatteryManager` estimate (no ODPM on this device).

14.7 Findings

Table 8: S1 – summary metrics.

Metric	Value
Trace duration	26.4 s
CPU peak (cold start)	50–60%
CPU avg (idle)	<5%
Total threads	62
Main thread % Running	3.97%
Main thread % Sleeping	74.39%
Total RSS	128 MB
Java/Kotlin heap	32 MB
Frames	131
Janky frames	7 (5.34%)
Longest frame	1.06 s (cold start)
<code>bindApplication</code> time	476.6 ms
<code>activityStart</code> time	339.5 ms
Battery class	Light
ODPM power rails	N/A (no Pixel 6+)

Strengths. The main thread is essentially idle (3.97% Running, 74.39% Sleeping). Java heap stays near 32 MB with no leak signals. GPU pressure is low.

Weaknesses. Three of the seven janky frames (2080582, 2080585, 2080722) sit inside the fragment transition to `DeviceFragment` and show 53–72 ms of application work per frame. The cost comes from inflating the plugin card list on the main thread.

Proposed optimisation (rubric point 4.c). Migrate the plugin card list in `DeviceFragment` from the current `Compose + ViewBinding` mix to a single `LazyColumn`. `LazyColumn` composes only the visible items, so the per-frame measure/layout cost would scale with what the user sees, not with the total number of enabled plugins.

15 Scenario S2 – Large file transfer with network failure and leak detection

15.1 Scenario definition

S2 has three objectives: (i) characterise the application during an active 436 MB transfer (CPU, memory pressure, parallelism in `BackgroundJobHandler`, the T3 strategy from Part 3); (ii) verify anti-pattern A1 from Part 3 (*File transfer has no resume; partial files are deleted on failure*) by forcing a network drop mid-transfer; and (iii) follow up on whatever emerges from the heap dumps. The third objective produced an unplanned but significant finding: a real memory leak in `DeviceFragment` that we later reported and patched upstream.

Test file. 436 MB .zip, sent from the paired desktop client (JunF3rm4t21) to the phone.

Captures. Three heap dumps taken at well-defined points (idle, mid-transfer at ~50%, immediately after the induced Wi-Fi drop) plus one extra heap dump after applying the fix. A full System Trace of a successful transfer was planned but not captured; the static analysis of T3 in Part 3 plus the S1 trace already cover threading behaviour for this code path.

15.2 CPU consumption and threading

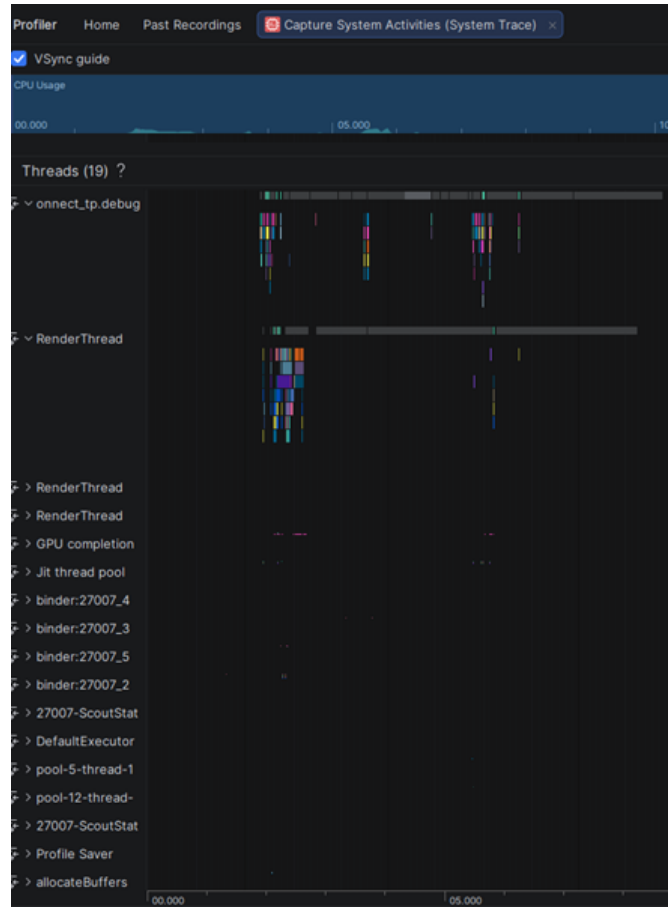


Figure 18: S2.1 – Threads active during the transfer (19 in total). The main thread is mostly idle; transfer work runs on `pool-5-thread-1` and friends.

The transfer is offloaded to a fixed-size pool (5 threads, as documented for T3 in Section 12.3 of Part 3). The main thread shows no blocking work during the transfer. Total thread count remained at 19, lower than S1 because no plugin transitions are happening.

15.3 Memory management

We took three heap dumps. The progression is the most informative metric in this scenario.

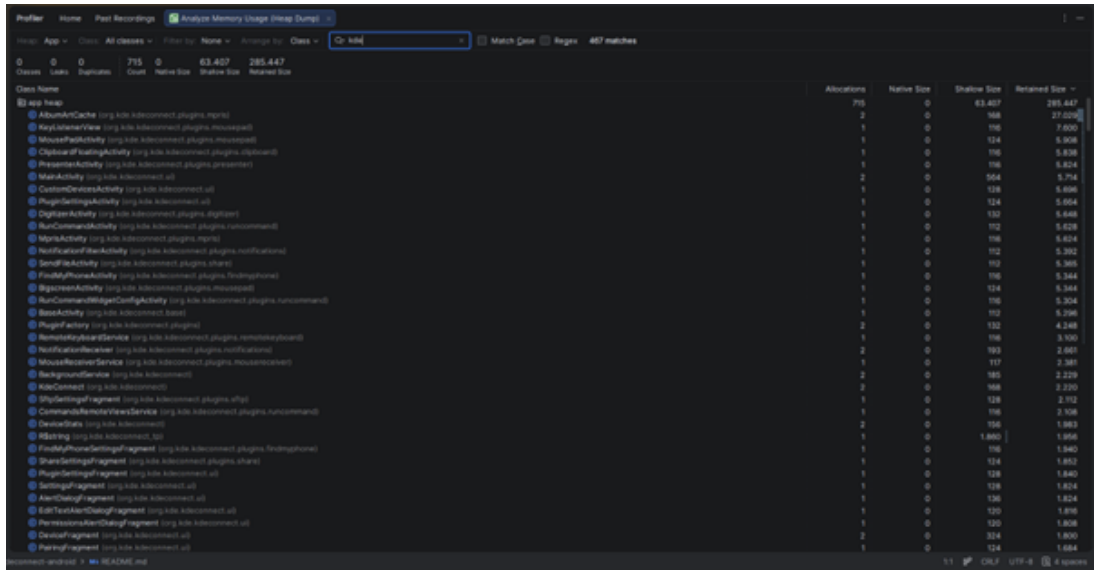


Figure 19: S2.2 – Heap dump 1: idle, before the transfer.

Phase 1 – idle. Retained size 285 KB. No leak markers from the Profiler.

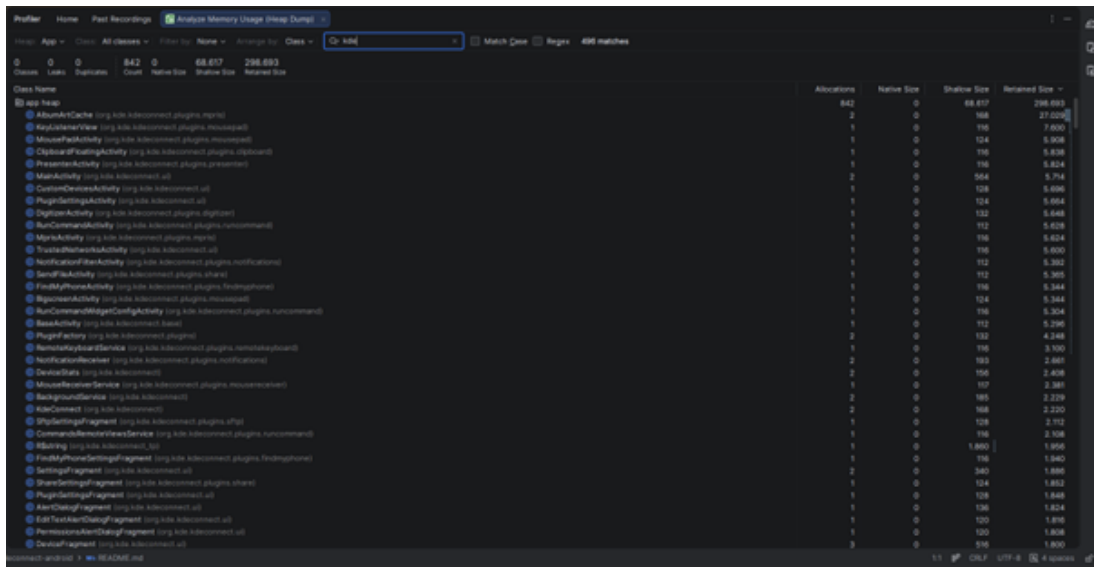


Figure 20: S2.3 – Heap dump 2: during the transfer.

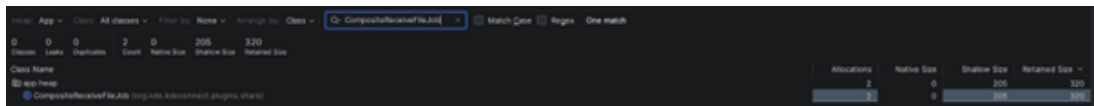


Figure 21: S2.4 – Heap dump 2 filtered by CompositeReceiveFileJob. Two live instances drive the entire download.

Class Name	Allocations	Native Size	Shallow Size	Retained Size
byte[]	18,055	2,679,622	624	624

Figure 22: S2.5 – Heap dump 2 filtered by `byte[]`. 18,055 short-lived allocations totalling 2.6 MB shallow size, only 624 bytes retained.

Phase 2 – mid-transfer. Retained heap during the transfer grew by only 13 KB compared to idle (285 KB → 298 KB). This is the key finding for memory management: a 436 MB file does not accumulate in the Java heap because the receive path is implemented as a streaming pipeline. The cost shows up as allocation churn – 18,055 `byte[]` allocations – not as retained memory, and the GC reclaims them as the transfer progresses.



Figure 23: S2.6 – System notification after the induced Wi-Fi drop: *Error al recibir archivo desde ARTEMIS*.

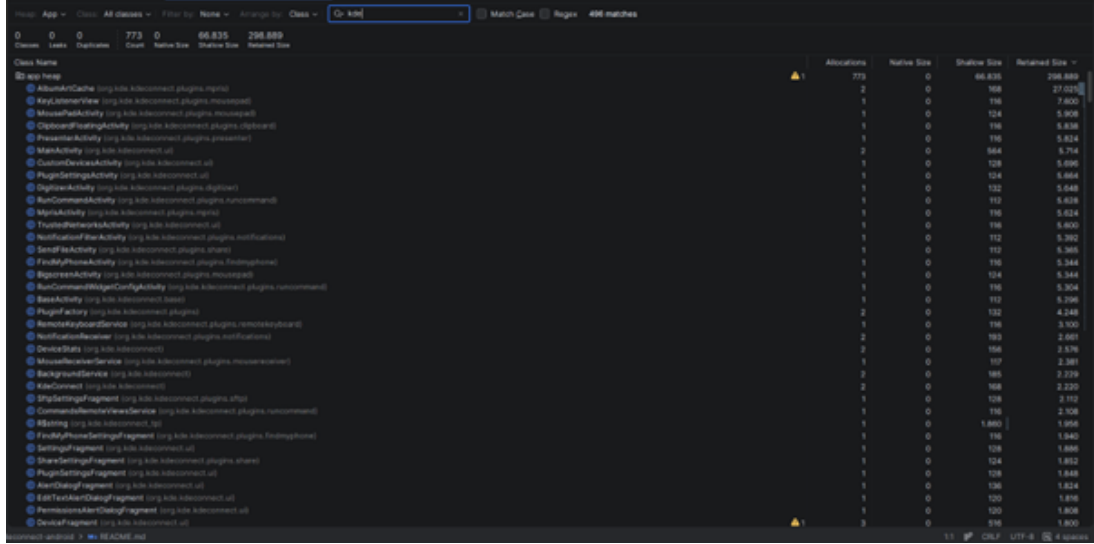


Figure 24: S2.7 – Heap dump 3: post-failure. Retained size returned to 298 KB.

Table 9: S2 – retained heap and leak markers across the three phases.

Phase	Retained heap	Leak markers (RAM)
Idle	285 KB	None
During transfer	298.7 KB (+13 KB)	None
Post-failure	298.9 KB	None in RAM (but 269 MB partial file on disk)

Phase 3 – induced Wi-Fi drop. The Java heap is clean after the failure, but a manual inspection of Downloads/ showed that the 269 MB partial file that had been written so far was *not* deleted. This refines the description of A1 from Part 3: the on-disk cleanup branch does not run for the network-drop error path, so the partial file persists as user-visible garbage.

15.4 GPU rendering analysis

The transfer is I/O and CPU bound. No animation runs during the transfer and the progress UI did not produce any janky frame; GPU analysis is not informative for this scenario.

15.5 Overdraw

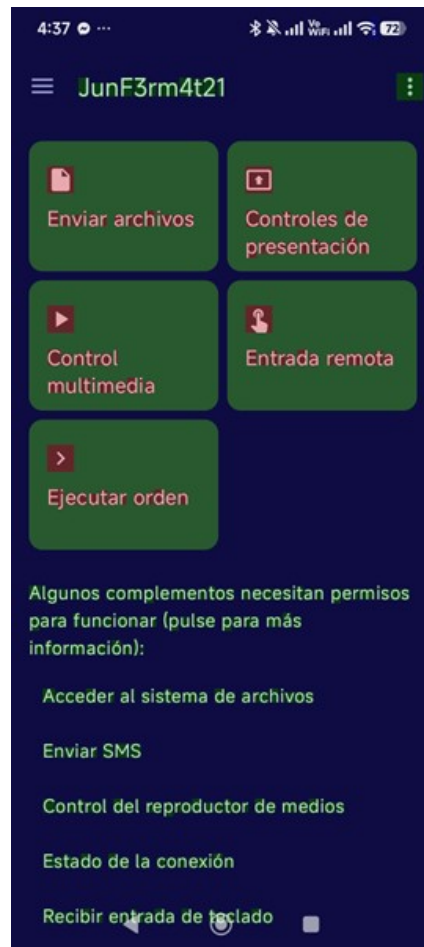


Figure 25: S2.8 – Overdraw on `DeviceFragment` (device dashboard). Blue base (1x), green buttons (2x), pink text (3x). No red regions.

The dashboard composes within the expected 1x–3x range; the only 3x regions are text drawn on top of card backgrounds, which is unavoidable.

15.6 Power consumption

Classified as *Light to Moderate* by the Profiler. The on-device battery indicator dropped about 3% over the full scenario, attributable to a combination of CPU, sustained Wi-Fi reception and disk writes.

15.7 Memory leak in `DeviceFragment`: detection, root cause and fix

While inspecting heap dump 3, the Profiler showed a leak marker (*There is 1 leak*) next to `DeviceFragment`. The marker was *not* present in heap dumps 1 and 2, which made the diff easy to read.

Patch. The least invasive fix is to make the registration idempotent at the call site by removing before adding. This keeps the code local to `DeviceFragment` and changes no behaviour for callers that register exactly once:

Listing 33: `DeviceFragment.onViewCreated`: defensive remove-before-add.

```

1 device?.apply {
2     mActivity?.supportActionBar?.title = name
3     removePairingCallback(pairingCallback)           // new
4     addPairingCallback(pairingCallback)
5     removePluginsChangedListener(pluginsChangedListener) // new
6     addPluginsChangedListener(pluginsChangedListener)
7 }

```

Empirical verification.

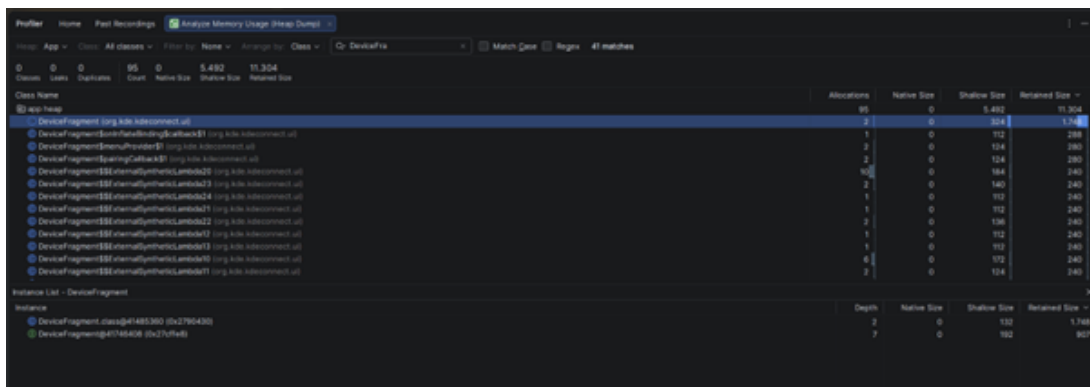


Figure 28: S2.12 – Heap dump after applying the patch: a single live `DeviceFragment`, zero leak markers.

Table 11: S2 – pre-fix vs post-fix leak comparison.

Metric	Pre-fix (HD3)	Post-fix
Leak markers from the Profiler	1	0
Warning on <code>DeviceFragment</code>	Yes	No
Live instances after navigation + GC	2 (1 active + 1 leaked)	1 (active only)

15.8 Upstream contribution to KDE Connect

The leak was significant enough to escalate through the official KDE workflow.

Duplicate search. KDE Bugzilla was searched without finding a specific report for this leak. Bug 452410 (duplicate notifications) was identified as related but distinct.

Bug report.

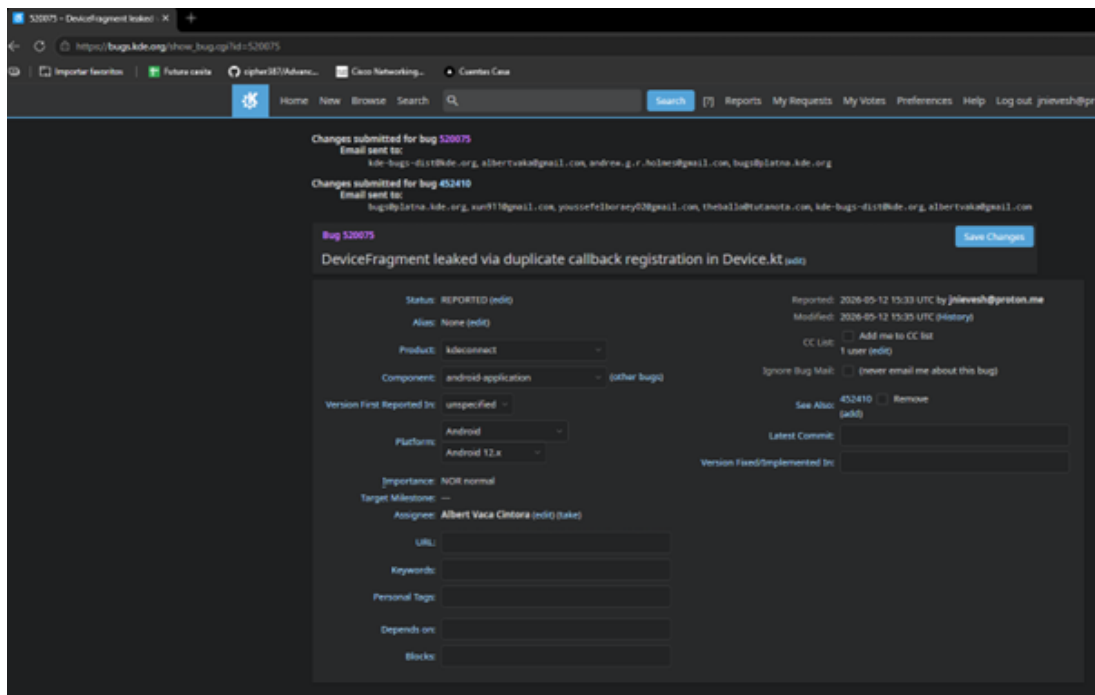


Figure 29: S2.11 – KDE Bugzilla 520075 in REPORTED state, assigned to the main maintainer.

Table 12: S2 – bug report metadata.

Field	Value
Bug ID	520075
URL	https://bugs.kde.org/show_bug.cgi?id=520075
Product	kdeconnect
Component	android-application
Status	REPORTED
Assignee	Albert Vaca Cintora (main maintainer)

Merge request.

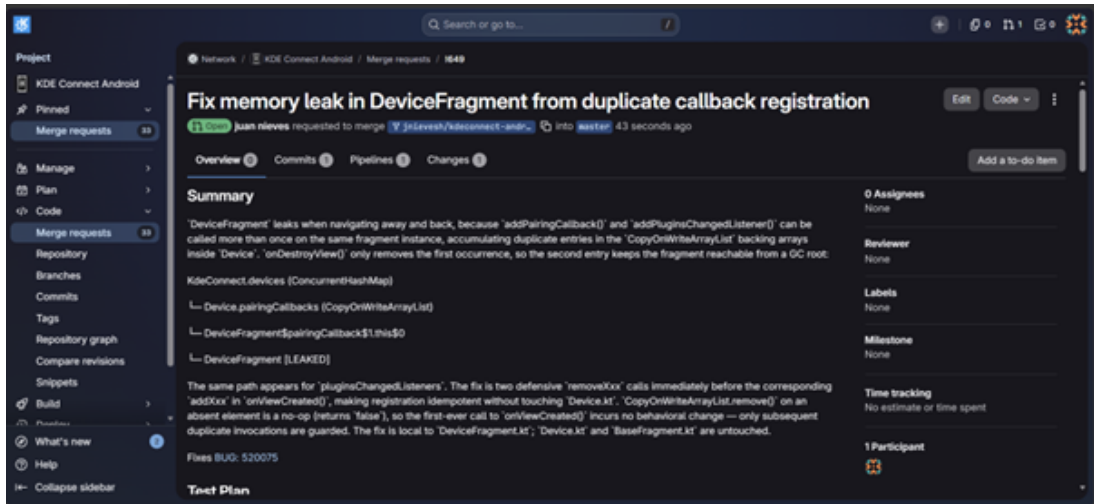


Figure 30: S2.13 – Merge request !649 on invent.kde.org, status Open, with the patch shown above.

Table 13: S2 – merge request metadata.

Field	Value
MR ID	!649
URL	https://invent.kde.org/network/kdeconnect-android/-/merge_requests/649
Source branch	jnievesh/kdeconnect-android:fix/devicefragment-listener-leak
Target branch	network/kdeconnect-android:master
Status	Open
Files changed	1 (DeviceFragment.kt)
Lines	+2 / -0

15.9 Findings

Table 14: S2 – summary metrics.

Metric	Value
Test file	436 MB (.zip)
Event type	Transfer + induced network drop
Java heap, idle (filtered)	~285 KB
Java heap, during transfer (filtered)	~298 KB
Heap delta during transfer	+13 KB (no in-memory file buffer)
byte[] allocation churn	18,055 / 2.6 MB shallow / 624 B retained
Partial file after failure	269 MB persisted in Downloads/ (refines A1)
Memory leaks detected	1 (DeviceFragment)
Root cause	Duplicate registration in Device.pairingCallbacks
Bug report	KDE Bugzilla 520075
Merge request	invent.kde.org !649
Post-fix verification	0 leak markers

Strengths. The streaming I/O design works as intended: a 436 MB transfer adds only 13 KB to the Java heap. The high `byte[]` churn is reclaimed correctly. The listener lifecycle in `DeviceFragment` is partially correct (removes happen in `onDestroyView`); the bug is the duplication, not the omission.

Weaknesses. Two findings here. (i) A1 from Part 3 is refined: the partial file is not deleted on the error path observed (269 MB left as user-visible garbage). (ii) The `DeviceFragment` leak documented above. (iii) The `byte[]` churn rate suggests there is no reusable buffer pool for the receive path.

Proposed optimisations (rubric point 4.c).

1. The leak fix described above, already submitted as MR !649.
2. Implement transfer resume to refine A1: persist (`peerId`, `fileKey`, `bytesReceived`) in `SharedPreferences` and send a resume command after reconnection.
3. Introduce a buffer pool for I/O: reuse `byte[]` arrays instead of allocating one per chunk, reducing GC pressure.

16 Scenario S3 – Screen rotation (configuration change)

16.1 Scenario definition

S3 evaluates how the application handles configuration changes (vertical $\leftrightarrow$ horizontal), measuring memory management (accumulation of fragments or activities), GPU rendering during the rotation transition, overdraw on the rotated views, and thread behaviour.

The flow opens KDE Connect on the device list, forces a rotation via system auto-rotate, waits two seconds, rotates back, and repeats the cycle ten times to surface any accumulating retention.

16.2 CPU consumption and threading

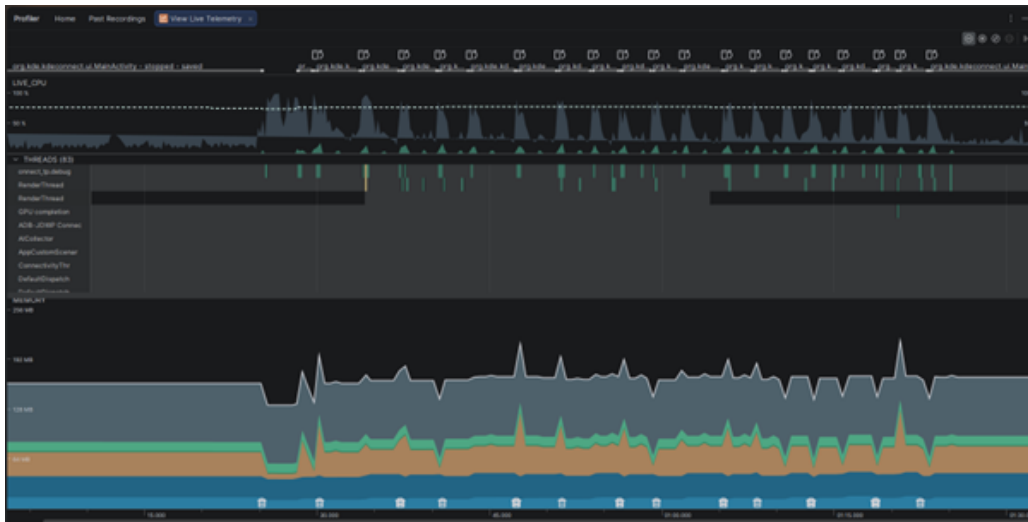


Figure 31: S3.1 – CPU usage across the ten rotations. Short peaks (30–40%) at each rotation correspond to view re-layout and re-bind.

The main thread alternates between short `Running` bursts (<200 ms per rotation) for the redraw and `Sleeping` the rest of the time. No long `Runnable` queues were observed. Total thread count was 83; extra threads created during the first rotations were released afterwards.

Top events during a typical rotation: `onConfigurationChanged` (15–25 ms CPU), `measure/layout` of the root activity (30–50 ms), `draw` (10–15 ms).

16.3 Memory management



Figure 32: S3.2 – Live memory telemetry during the ten rotations. RSS climbed from 161 MB to a peak of 212 MB, then settled at 170 MB after the final rotation and a forced GC.

Table 15: S3 – memory snapshots.

Moment	Total memory (RSS)	Observation
Before rotating	161 MB	Baseline
Peak during rotations	212 MB	Transient increase
After 10 rotations + GC	170 MB	Near-baseline recovery

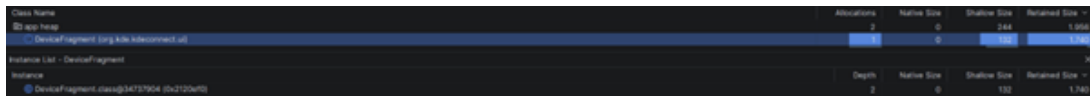


Figure 33: S3.3 – Heap dump before rotating, filtered by `DeviceFragment`: one instance.

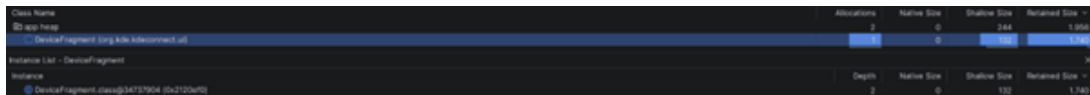


Figure 34: S3.4 – Heap dump after ten rotations and a forced GC, filtered by `DeviceFragment`: still one instance.

Table 16: S3 – instance counts in the heap dumps.

Class	Before	After	Retained	Leak?
<code>DeviceFragment</code>	1	1	~1.8 KB	No
<code>MainActivity</code>	1	1	~5.7 KB	No
<code>byte[]</code>	~18,000	~18,100	~600 B	No (normal churn)

No rotation-induced leak. RSS rises transiently during configuration changes (the view tree is recreated) and the GC reclaims it. No orphan instances of `DeviceFragment` or `MainActivity` remain.

16.4 GPU rendering analysis

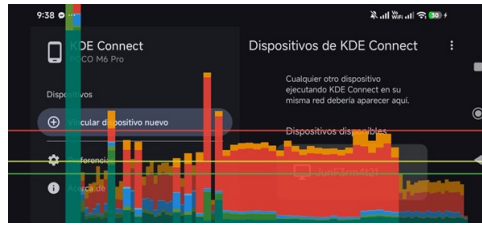


Figure 35: S3.5 – GPU rendering bars during rotation. A short burst of red/orange bars exceeds the 16 ms line at the exact rotation instant.

The red peaks are expected behaviour during a configuration change: the activity recreates its view tree to recompute landscape positions while animating the rotation. Each red burst lasts a fraction of a second and the application returns to 60 FPS immediately after.

A System Trace over a 10 s window covering three rotations produced the numbers in Table 17.

Table 17: S3 – rendering metrics over a 10 s window with three rotations.

Metric	Value
Total frames	287
Janky frames	4
Jank rate	1.39%
Longest frame	22.3 ms

16.5 Overdraw

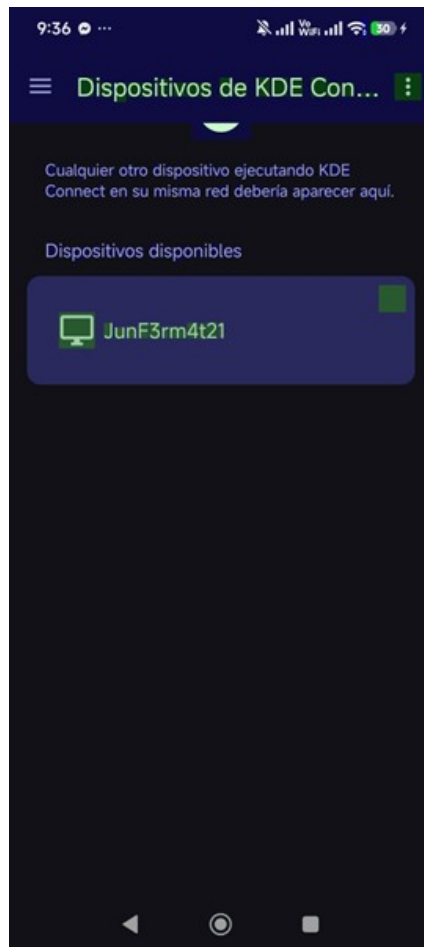


Figure 36: S3.6 – Overdraw in portrait: blue (1x) dominates.



Figure 37: S3.7 – Overdraw in landscape: same pattern as portrait, with localised green (2x) on list items.

Overdraw remains in the 1x–2x range in both orientations. No regressions on rotation.

16.6 Power consumption



Figure 38: S3.8 – Battery panel during the ten rotations. Stable current with small ripples; no sustained peaks.

Battery dropped from 100% to 99% over the scenario. Classification: *Light*. `BatteryManager` estimate, no ODPM available on the device.

16.7 Findings

Table 18: S3 – summary metrics.

Metric	Value
Scenario duration	~2 min
Number of rotations	10
Initial RSS	161 MB
Peak RSS during rotations	212 MB
Final RSS (post-GC)	170 MB
Memory leak	No
Total frames (10 s window)	287
Janky frames	4 (1.39%)
Longest frame	22.3 ms
Overdraw (dominant colour)	Blue / light green
Total threads	83
Battery class	Light

Strengths. No rotation-induced leak: heap dumps show unique `DeviceFragment` and `MainActivity` instances before and after. Jank rate is low (1.39%) and confined to the rotation instants. Overdraw does not regress on rotation. The main thread is not blocked.

Weaknesses. No significant weakness was observed in this scenario.

Proposed optimisation (rubric point 4.c). The peak RSS during the rotation cycle could be reduced further by moving more of the per-screen state into an activity-scoped `ViewModel`, so that the data does not need to be re-derived on each recreation. This is a refinement rather than a defect fix.

17 Cross-scenario synthesis: GC, threading and main-thread contention

RAM consumption across scenarios. Table 19 compares RSS observed in each scenario. The pattern is consistent: the idle baseline lives near 128 MB RSS, file transfers do not push the heap at all because the receive path is implemented as a streaming pipeline (only +13 KB delta in S2), and a configuration change is the only scenario that visibly bumps RSS during the workload (because the view tree is recreated). In S3 the GC reclaims the increase back to within 10 MB of the pre-rotation baseline.

Table 19: RAM consumption per scenario (RSS reported by the Profiler).

Scenario	Baseline RSS	Peak RSS	Notes
S1 – Cold start	—	128 MB	First-launch baseline; flat thereafter
S2 – 436 MB transfer	128 MB	~128 MB	Java heap delta only +13 KB (streaming I/O)
S3 – Ten rotations	161 MB	212 MB	Recovers to 170 MB post-GC

GC frequency and allocation patterns. Across the three scenarios, GC activity scales with allocation rate, as expected. S1 (idle UI) showed sparse GC events with no long pauses, consistent with the flat Java heap line at ~32 MB. S2 is the most allocation-intensive scenario: 18,055 `byte[]` allocations during the transfer produced steady GC activity, but with a 13 KB retained-heap delta, indicating the GC kept up with the churn. S3 produced a transient peak in RSS (161 MB → 212 MB) during the ten rotations, fully reclaimed after a forced GC (170 MB). No deep allocation pattern persists after the workload ends in any scenario except for the S2 `DeviceFragment` leak that we documented and patched.

Main-thread contention. We did not observe any lock-held-on-main behaviour. The S1 main-thread state distribution (74.39% Sleeping, 3.97% Running, 0.36% Runnable) is dominated by VSYNC waits, not by contention. S3 confirms the same pattern under repeated configuration changes. The single observed source of main-thread cost is the synchronous inflation of plugin cards in `DeviceFragment` (53–72 ms per frame in three of the S1 janky frames), which is layout work rather than lock contention.

How threading contributes to performance. The threading map documented in Part 3 (T1–T5) directly explains the observed metrics. T1 (the shared cached pool) is responsible for the brief CPU bursts visible in S1 and S3 during fragment lifecycle work. T3 (the fixed pool with `Futures`, capped at five threads for share work) is responsible for keeping S2 within 19 active threads and for the absence of main-thread blocking during the transfer. T5 (lock-free and lock-biased containers, including `CopyOnWriteArrayList` for the callback lists) explains both a strength and a weakness: it gives cheap iteration from worker threads, but it is also the data structure that surfaced the duplicate-registration leak in `DeviceFragment`. The fix in MR !649 keeps the lock-free benefit while making the registration idempotent.

18 Micro-optimisations and detected anti-patterns

This section identifies the micro-optimisations present in the code (rubric point 4.a–b) and the parts of the code that would benefit from further optimisation (rubric point 4.c). Each pattern has a numeric label (MO1–MO5) to make cross-references compact. The anti-pattern that closes the section continues the A-numbering from Part 3 (A4).

18.1 MO1. MD5 hashing for cache keys

What. The disk cache layer turns an arbitrary image URL into a fixed-length, filesystem-safe key by hashing it with MD5.

Location. `AlbumArtCache.kt`, around line 115.

Listing 34: `AlbumArtCache.kt`: MD5-based disk cache key.

```

1  /**
2   * The disk cache requires mostly alphanumeric characters, and at most
3   * 64 characters. So hash the url to get a valid key.
```

```

4  */
5  private fun urlToDiskCacheKey(url: String): String {
6      return MessageDigest.getInstance("MD5").digest(url.toByteArray())
7          .joinToString(separator = "") { String.format("%02x", it) }
8  }

```

Why this is a micro-optimisation. The functional requirement is just *be a valid filename*, but the choice of MD5 turns an unbounded-length URL with disallowed characters into a fixed 32-character hex string that the disk cache can index in $O(1)$. It is a one-line change with bounded cost that prevents a class of runtime errors.

Purpose. Filesystem compatibility on Android (no special characters, length limit) and constant-time indexing of cache entries.

Refinement opportunity (4.c). The hex encoding uses `String.format("%02x", it)` inside a loop, which allocates a format object per byte. Replacing this with a direct nibble-to-hex mapping over a preallocated `CharArray` removes the per-byte allocation and is several times faster, with no behavioural change.

18.2 MO2. Lock-biased containers (`CopyOnWriteArrayList`) for read-heavy listener lists

What. Listener lists that are iterated frequently from worker threads but mutated rarely are stored in `CopyOnWriteArrayList` instead of an `ArrayList` guarded by `synchronized`.

Location. `Device.kt` (`pairingCallbacks`, `pluginsChangedListeners`), and similar occurrences across the codebase (already documented in Part 3 under T5).

Listing 35: `Device.kt`: listener list backed by `CopyOnWriteArrayList`.

```

1 private val pairingCallbacks = CopyOnWriteArrayList<PairingCallback>()

```

Why this is a micro-optimisation. The container choice removes the need for explicit synchronisation on the iteration path, which is the hot path (every state change iterates the list to notify callbacks). The mutation path pays the cost of a copy, but mutation is rare.

Purpose. Avoid blocking worker threads on a monitor, and avoid `ConcurrentModificationException` when a callback unsubscribes itself during iteration.

18.3 MO3. Bounded in-memory cache (`LruCache`) for decoded bitmaps

What. A 10-entry `LruCache` stores decoded album-art bitmaps in memory, with the failure-to-fetch state stored alongside successful entries.

Location. `AlbumArtCache.kt`, class field.

Listing 36: `AlbumArtCache.kt`: bounded in-memory bitmap cache.

```

1 /**
2  * An in-memory cache for album art bitmaps. Holds at most 10 entries
3  * (to prevent too much memory usage). Also remembers failure to fetch
4  * urls.
5  */
6 private val memoryCache = LruCache<String, MemoryCacheItem>(10)

```

Why this is a micro-optimisation. It replaces a hand-written cache with an LRU that has a hard upper bound, so memory consumption is deterministic. Without the bound, the same code would be a `HashMap` that grows for the duration of the session.

Purpose. Avoid re-decoding bitmaps that the user re-visits (speed) and prevent the bitmap cache from triggering `OutOfMemoryError` on low-memory devices.

18.4 MO4. Lazy plugin loading via annotations

What. Each plugin class is marked with `@LoadablePlugin`. Plugin discovery is annotation-driven, so plugin objects are constructed on demand rather than at process start.

Location. All plugin classes, for example `SMSPlugin.kt`.

Listing 37: `SMSPlugin.kt`: annotation-driven registration.

```
1 @LoadablePlugin
2 @SuppressWarnings("InlinedApi")
3 class SMSPlugin : Plugin() {
4     // ...
5 }
```

Why this is a micro-optimisation. The annotation replaces what would otherwise be a static registration list in application start-up code. Plugins that the user does not enable do not pay an instantiation cost.

Purpose. Reduce cold-start CPU and RAM. This is consistent with the S1 observation that `bindApplication` is 477 ms wall clock but only 83 ms CPU: the framework is doing the work, the application is not adding linear plugin construction to it.

18.5 MO5. Listener reuse in mDNS discovery

What. The class that owns the discovery lifecycle registers itself (`this`) as the mDNS callback handler instead of allocating an anonymous inner class.

Location. `LanLinkProvider.java`, constructor.

Listing 38: `LanLinkProvider.java`: registration via `this`.

```
1 public LanLinkProvider(Context context) {
2     this.context = context;
3     this.mdnsDiscovery = new MdnsDiscovery(context, this);
4 }
```

Why this is a micro-optimisation. Passing `this` avoids one extra heap allocation per discovery start, and the same instance also avoids retaining a captured outer reference. In the discovery code path this is invoked every time network conditions change, so the cumulative effect is non-trivial.

Purpose. Reduce GC pressure on the discovery path and keep the memory footprint flat during repeated reconnects.

18.6 A4. Unbounded cached thread pool (anti-pattern, continues A-numbering from Part 3)

What. The shared `ThreadHelper` uses `Executors.newCachedThreadPool()`, which has no upper bound on thread count.

Location. `src/main/java/org/kde/kdeconnect/helpers/ThreadHelper.kt`, line 16.

Listing 39: ThreadHelper.kt: the current implementation.

```
1 object ThreadHelper {
2     private val executor: ExecutorService = Executors.
        newCachedThreadPool()
3
4     @JvmStatic
5     fun execute(command: Runnable) = executor.execute(command)
6 }
```

Why this is an anti-pattern on mobile. A cached pool is appropriate on a server with abundant memory and a roughly bounded request rate. On Android, each thread reserves a stack (~1 MB), the OS schedules every thread it sees, and a sudden burst of events (concurrent packet arrival, plugin notification, broadcast) can spawn many threads at once. We did not observe a critical burst in S1–S3 (S1 peaked at 62 threads, S2 stayed at 19, S3 stayed around 83), but the structural risk on a low-memory device or under a network burst remains.

Proposed fix (4.c). Replace the cached pool with an explicit `ThreadPoolExecutor` that has a hard ceiling and a bounded queue, or migrate to `Dispatchers.IO` from Kotlin coroutines (already used elsewhere in the codebase, see T2 in Part 3).

Listing 40: Proposed replacement for ThreadHelper.kt.

```
1 // Bounded pool: 2 core threads, 4 maximum.
2 private val executor: ExecutorService = ThreadPoolExecutor(
3     2, 4, 60L, TimeUnit.SECONDS,
4     LinkedBlockingQueue()
5 )
```

Expected effect. Thread-count memory cost capped at 4 MB (vs. unbounded). `OutOfMemoryError` risk under network bursts eliminated. CPU context switching reduced under load, which translates into more stable frame timings on lower-end devices.